Notebook ICPC
La Tasita

Team: UPC - Turistas

Integrantes

spigi
Jean_Pool
Santi2007939
Chatito17

Índice

Dynamic Programming

- Convex Hull Trick 3
- Data Structure Optimization 4
- Digit Dp 5
- Divide And Conquer 5
- Knapsack 6
- Knapsack Binary 7
- Knapsack Re 7
- Knapsack Subset Sum 7
- Knuth 8
- LCS 8
- LIS 8
- Lisfast 9
- Maximum Subarray 10
- Subset Sum Bitset 10

Data Structures

- Binary Lifting Weighted 11
- Binary Lifting 11
- DSU Rollback 12
- DSU 12
- Fenwick Tree 2D Range 13
- Fenwick Tree 2D 13
- Fenwick Tree Range 13
- Fenwick Tree 13
- HLD 14
- Ordered Set 15
- Persistent Segment Tree 16
- Segment Tree Lazy 17
- Segment Tree 17
- Small To Large Map 18
- Small To Large Santi 19
- Small To Large Set 20
- Sparse Table 21
- Two Pointers 21

Geometry

- Convex Hull 22
- Vector 23

Graphs

- 2-SAT 24
- Bellman Ford 25
- Cd 26
- Centroid Decomposition 28
- Dijkstra 29
- Floyd Warshall 29
- Ford Fulkerson 30
- Functional Graph 31
- Hungarian 33
- Kosaraju Scc 34
- Kruskal 36
- Topological Sort 37

Math

- Binary Exponentiation 38
- Expected Value 38
- FFT 39
- Matrix Exponentiation 40
- NTT 41
- Pollard Rho 43
- Sieve 44

Strings

Aho Corasick 45
Bit Trie 46
Double Hash 46
Hash 47
KMP Automaton 47
KMP 49
Rolling Hash 49
String Hashing 50
Suffix Array 51
Suffix Tree 52
Trie 55
Z Function 55

Otros

Plantilla 56

Dynamic Programming

Convex Hull Trick

```
// Convex Hull Trick (CHT) - O(1) amortizado por insercion y
query
// Optimiza: dp[i] = min/max ( m_j * x_i + c_j )
// Condicion: Pendientes 'm' insertadas en orden MONOTONO
(creciente/decreciente).
// Para queries 'x' monotonos: query() usa dos punteros O(1).
// Para queries 'x' arbitrarios: query_bin() usa busqueda
binaria O(log N).

struct Line {
    ll m, c;
    ll eval(ll x) const { return m * x + c; }
};

// CHT para MINIMIZAR (m decreciente, queries x crecientes)
// Para MAXIMIZAR: invertir signos o usar m creciente y
cambiar condicion is_bad
struct CHT {
    vector<Line> hull;
    int ptr = 0;

    // Retorna true si l2 es redundante entre l1 y l3
    bool is_bad(const Line& l1, const Line& l2, const Line&
l3) {
        // Interseccion(l1, l2) >= Interseccion(l2, l3) usando
__int128 para evitar overflow
        return (__int128)(l1.c - l2.c) * (l3.m - l2.m) >=
(__int128)(l2.c - l3.c) * (l2.m - l1.m);
    }

    void add(ll m, ll c) {
        Line l = {m, c};
        while (sz(hull) >= 2 && is_bad(hull[sz(hull) - 2],
hull.back(), l)) {
            hull.pop_back();
        }
        hull.pb(l);
        if (ptr >= sz(hull)) ptr = sz(hull) - 1;
    }
```

```
    // Query O(1) amortizado (requiere x monotonamente
creciente)
    ll query(ll x) {
        if (hull.empty()) return LINF;
        while (ptr + 1 < sz(hull) && hull[ptr + 1].eval(x) <=
hull[ptr].eval(x)) {
            ++ptr;
        }
        return hull[ptr].eval(x);
    }

    // Query O(log N) para x arbitrarios (no requiere que x
sea monotono)
    ll query_bin(ll x) {
        if (hull.empty()) return LINF;
        int l = 0, r = sz(hull) - 1;
        while (l < r) {
            int mid = l + (r - l) / 2;
            if (hull[mid + 1].eval(x) <= hull[mid].eval(x)) l
= mid + 1;
            else r = mid;
        }
        return hull[l].eval(x);
    }
};
```

Data Structure Optimization

```
// DP Optimization with Data Structures (Segment Tree /
Fenwick) - O(N log N)
// Optimiza transiciones: dp[i] = max_{j < i, L <= val[j] <=
R} { dp[j] } + cost[i]

// ─── 1. Compresion de Coordenadas
──────────────────────────────────────
// Mapea valores arbitrarios a [0, sz(vals)-1]
template<class T>
vi compress(const vector<T>& a) {
    vector<T> vals = a;
    sort(all(vals));
    vals.erase(unique(all(vals)), vals.end());
    vi res(sz(a));
    rep(i, sz(a)) {
        res[i] = lower_bound(all(vals), a[i]) - vals.begin();
    }
    return res;
}

// ─── 2. Segment Tree para Maximos en Rango
──────────────────────────────
struct SegTreeDP {
    int n;
    vll tree;
    SegTreeDP(int n) : n(n), tree(4 * n, 0) {}

    void update(int pos, ll val, int node, int l, int r) {
        if (l == r) {
            ckmax(tree[node], val);
            return;
        }
        int mid = l + (r - l) / 2;
        if (pos <= mid) update(pos, val, 2 * node, l, mid);
        else update(pos, val, 2 * node + 1, mid + 1, r);
        tree[node] = max(tree[2 * node], tree[2 * node + 1]);
    }
    void update(int pos, ll val) { update(pos, val, 1, 0, n -
1); }

    ll query(int ql, int qr, int node, int l, int r) {
        if (ql > r || qr < l) return 0; // valor neutro
        if (ql <= l && r <= qr) return tree[node];
        int mid = l + (r - l) / 2;
        return max(query(ql, qr, 2 * node, l, mid),
                   query(ql, qr, 2 * node + 1, mid + 1, r));
    }
    ll query(int ql, int qr) {
        if (ql > qr) return 0;
        return query(ql, qr, 1, 0, n - 1);
    }
};

// ─── 3. Fenwick de Maximos (O(log N) - Solo para Prefijos
[0..idx]) ──────────
struct BITMax {
    int n;
    vll bit;
    BITMax(int n) : n(n), bit(n + 1, 0) {}
```

```
    void update(int i, ll val) {
        for (++i; i <= n; i += i & -i) ckmax(bit[i], val);
    }
    ll query(int i) { // maximo en [0..i]
        ll res = 0;
        for (++i; i > 0; i -= i & -i) ckmax(res, bit[i]);
        return res;
    }
};

// Ejemplo de uso (LIS Ponderado / Flowers):
// vi comp = compress(alturas);
// SegTreeDP st(sz(comp));
// rep(i, n) {
//     ll best_prev = st.query(0, comp[i] - 1);
//     st.update(comp[i], best_prev + bellezas[i]);
// }
// ll ans = st.query(0, sz(comp) - 1);
```

Digit Dp

```
// Digit DP Template - Complejidad O(Longitud * Estados * 10)
// Estados tipicos: (pos, tight, leading_zero,
estado_personalizado)

// ─── 1. Enfoque Estandar f(R) - f(L - 1)
──────────────────────────────────
string S;
ll memo[20][2][2][100]; // Ajustar dimensiones del estado
personalizado

ll digitDP(int pos, bool tight, bool lz, int state) {
    if (pos == sz(S)) {
        // Retornar 1 si el 'state' es valido, 0 si no
        return 1;
    }
    ll& ans = memo[pos][tight][lz][state];
    if (ans != -1) return ans;

    ans = 0;
    int limit = tight ? (S[pos] - '0') : 9;

    for (int dig = 0; dig <= limit; ++dig) {
        bool next_tight = tight && (dig == limit);
        bool next_lz = lz && (dig == 0);

        // Actualizar estado personalizado (ejemplo: suma de
digitos, bitmask, mod, etc.)
        int next_state = next_lz ? 0 : (state + dig);

        ans += digitDP(pos + 1, next_tight, next_lz,
next_state);
        // Si pide modulo: ans %= MOD;
    }
    return ans;
}

ll countValid(ll n) {
    if (n < 0) return 0;
    S = to_string(n);
    memset(memo, -1, sizeof(memo));
    return digitDP(0, true, true, 0);
}

// Rango [L, R]
ll solveRange(ll L, ll R) {
    return countValid(R) - countValid(L - 1);
}

// ─── 2. Enfoque Simultaneo [L, R] (Tight Inferior y
Superior) ─────────────────
// string A, B; // Rellenar A con '0' a la izquierda hasta que
sz(A) == sz(B)
// ll dp2[20][2][2][2]; // pos, tight_low, tight_high,
leading_zero
// int min_d = tight_low  ? (A[pos] - '0') : 0;
// int max_d = tight_high ? (B[pos] - '0') : 9;
// for (int d = min_d; d <= max_d; ++d) {
//     bool n_low  = tight_low  && (d == min_d);
//     bool n_high = tight_high && (d == max_d);
//     bool n_lz   = lz && (d == 0);
//     ...
// }
```

Divide And Conquer

```
// Divide and Conquer DP Optimization - O(K * N log N)
// Condicion: opt[k][i] <= opt[k][i+1] (e.g., Quadrangle
```

```
Inequality)
// Recurrencia: dp[k][i] = min_{j < i} { dp[k-1][j] + cost(j,
i) }

vll dp_prev, dp_curr;

// 1. Costo O(1) con formulas / prefijos (personalizar segun
problema)
ll cost(int j, int i) {
    // Ejemplo: (pref[i] - pref[j])^2
    // ll s = pref[i] - pref[j];
    // return s * s;
    return 0;
}

// 2. Variante: Costo con Ventana Dinamica / Two Pointers
estilo Mo
// int curL = 1, curR = 0; ll cur_cost = 0;
// void move_to(int L, int R) {
//     while (curR < R) add(++curR);
//     while (curL > L) add(--curL);
//     while (curR > R) remove(curR--);
//     while (curL < L) remove(curL++);
// }

void compute(int l, int r, int optL, int optR) {
    if (l > r) return;
    int mid = l + (r - l) / 2;
    int best_opt = -1;
    dp_curr[mid] = LINF;
    int limit = min(mid - 1, optR);

    for (int j = optL; j <= limit; ++j) {
        if (dp_prev[j] == LINF) continue;
        // Si se usa ventana dinamica: move_to(j + 1, mid);
        ll val = dp_prev[j] + cost(j, mid); // o dp_prev[j] +
cur_cost;
        if (val < dp_curr[mid]) {
            dp_curr[mid] = val;
            best_opt = j;
        }
    }

    compute(l, mid - 1, optL, best_opt);
    compute(mid + 1, r, best_opt, optR);
}

// Resuelve para K particiones en un arreglo de tamaño N (1-
indexed)
ll solveDCDP(int n, int k) {
    dp_prev.assign(n + 1, LINF);
    dp_curr.assign(n + 1, LINF);

    // Caso base k = 1
    dp_prev[0] = 0;
    rep1(i, n) dp_prev[i] = cost(0, i);

    // Transicion para k = 2..K
    for (int c = 2; c <= k; ++c) {
        compute(1, n, 0, n - 1);
        dp_prev = dp_curr;
    }
    return dp_prev[n];
}
```

Knapsack

```
// 0/1 Knapsack - O(N * W)
// items: {peso, valor}
ll knapsack(const vii& items, int W) {
    vll dp(W + 1, -1);
    dp[0] = 0;
    each(item, items) {
        int w = item.ff;
        ll v = item.ss;
        for (int j = W; j >= w; --j) {
            if (dp[j - w] != -1) {
                ckmax(dp[j], dp[j - w] + v);
            }
        }
    }
    ll max_v = 0;
    rep(j, W + 1) ckmax(max_v, dp[j]);
    return max_v;
}
```

Knapsack Binary

```
// Bounded Knapsack (Binary Splitting) - O(W * sum(log K))
ll knapsackBounded(const vi& pesos, const vll& valores, const
vi& copias, int W) {
    vii items;
    int n = sz(pesos);
    rep(i, n) {
        int k = copias[i], b = 1;
        while (k > 0) {
            int num = min(k, b);
            items.eb(pesos[i] * num, valores[i] * num);
            k -= num;
            b *= 2;
        }
    }
    vll dp(W + 1, -1);
    dp[0] = 0;
    each(item, items) {
        int w = item.ff;
        ll v = item.ss;
        for (int j = W; j >= w; --j) {
            if (dp[j - w] != -1) {
                ckmax(dp[j], dp[j - w] + v);
            }
        }
    }
    ll ans = 0;
    rep(j, W + 1) ckmax(ans, dp[j]);
    return ans;
}
```

Knapsack Re

```
// 0/1 Knapsack with Reconstruction - O(N * W)
// items: {peso, valor} -> retorna {max_valor,
indices_tomados}
pair<ll, vi> knapsackReconstruction(const vii& items, int W) {
    int n = sz(items);
    vvll dp(n + 1, vll(W + 1, 0));
    rep(i, n) {
        int w = items[i].ff;
        ll v = items[i].ss;
        rep(j, W + 1) {
            dp[i + 1][j] = dp[i][j];
            if (j >= w) ckmax(dp[i + 1][j], dp[i][j - w] + v);
        }
    }
    vi taken;
    int cur_w = W;
    for (int i = n - 1; i >= 0; --i) {
        int w = items[i].ff;
        ll v = items[i].ss;
        if (cur_w >= w && dp[i + 1][cur_w] == dp[i][cur_w - w]
+ v) {
            taken.pb(i);
            cur_w -= w;
        }
    }
    reverse(all(taken));
    return {dp[n][W], taken};
}
```

Knapsack Subset Sum

```
// Subset Sum with Equal Partition / Difference DP - O(N *
sum)
// Encuentra la máxima altura igual de dos subconjuntos
disjuntos (Tallest Billboard)
int knapsackDiferencia(const vi& arr) {
    int sum = 0;
    each(x, arr) sum += x;
    vi dp(sum + 1, -1);
    dp[0] = 0;

    each(h, arr) {
        vi next_dp = dp;
        rep(d, sum + 1) {
            if (dp[d] < 0) continue;
            // 1. No usar h (ya cubierto por next_dp = dp)
            // 2. Añadir al lado mayor
            if (d + h <= sum) ckmax(next_dp[d + h], dp[d]);
            // 3. Añadir al lado menor
            int diff = abs(d - h);
            ckmax(next_dp[diff], dp[d] + min(d, h));
        }
        dp = move(next_dp);
    }
```

```
    return dp[0];
}
```

Knuth

```
// Knuth's DP Optimization - O(N^2)
// Recurrencia: dp[i][j] = min_{i <= k < j} { dp[i][k] +
dp[k+1][j] } + cost(i, j)
// Condicion de optimo: opt[i][j-1] <= opt[i][j] <= opt[i+1]
[j]
// Requiere que cost(i, j) cumpla Quadrangle Inequality y
Monotonicidad

ll knuthDP(const vll& arr) {
    int n = sz(arr);
    // Prefijos para calcular costo de rango en O(1)
    vll pref(n + 1, 0);
    rep(i, n) pref[i + 1] = pref[i] + arr[i];

    auto cost = [&](int i, int j) -> ll {
        return pref[j + 1] - pref[i];
    };

    vvll dp(n, vll(n, 0));
    vvi opt(n, vi(n, 0));

    // Casos base longitud 1
    rep(i, n) {
        dp[i][i] = 0;
        opt[i][i] = i;
    }

    // Iterar por longitud del intervalo len = 2..N
    // O equivalentemente i de n-2 hasta 0, j de i+1 hasta n-1
    for (int len = 2; len <= n; ++len) {
        for (int i = 0; i <= n - len; ++i) {
            int j = i + len - 1;
            dp[i][j] = LINF;
            ll c = cost(i, j);

            int l_opt = opt[i][j - 1];
            int r_opt = (i + 1 <= j) ? opt[i + 1][j] : j - 1;
            r_opt = min(r_opt, j - 1);

            for (int k = l_opt; k <= r_opt; ++k) {
                ll val = dp[i][k] + dp[k + 1][j] + c;
                if (val < dp[i][j]) {
                    dp[i][j] = val;
                    opt[i][j] = k;
                }
            }
        }
    }
    return dp[0][n - 1];
}
```

LCS

```
// Longest Common Subsequence & Reconstruction - O(N * M)
pair<int, string> LCS(const string& s1, const string& s2) {
    int n = sz(s1), m = sz(s2);
    vvi dp(n + 1, vi(m + 1, 0));
    rep(i, n) rep(j, m) {
        if (s1[i] == s2[j]) dp[i + 1][j + 1] = dp[i][j] + 1;
        else dp[i + 1][j + 1] = max(dp[i][j + 1], dp[i + 1]
[j]);
    }
    string ans = "";
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1]) {
            ans += s1[i - 1];
            i--; j--;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    reverse(all(ans));
    return {dp[n][m], ans};
}
```

LIS

```
// O(N^2) Longest Increasing Subsequence
int LIS(const vi& arr) {
```

```
    int n = sz(arr);
    vi dp(n, 1);
    int res = 0;
    rep(i, n) {
        rep(j, i) {
            if (arr[j] < arr[i]) ckmax(dp[i], dp[j] + 1);
        }
        ckmax(res, dp[i]);
    }
    return res;
}

vi LISReconstruccion(const vi& arr) {
    int n = sz(arr);
    vi dp(n, 1), padre(n, -1);
    int res = 0, best_idx = -1;

    rep(i, n) {
        rep(j, i) {
            if (arr[j] < arr[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                padre[i] = j;
            }
        }
        if (dp[i] > res) {
            res = dp[i];
            best_idx = i;
        }
    }

    vi secuencia;
    for (int i = best_idx; i != -1; i = padre[i]) {
        secuencia.pb(arr[i]);
    }
    reverse(all(secuencia));
    return secuencia;
}
```

Lisfast

```
// LIS Fast O(N log N)
int LISFast(const vi& arr) {
    vi res;
    each(x, arr) {
        auto it = lower_bound(all(res), x); // upper_bound
para no estrictamente creciente
        if (it == res.end()) res.pb(x);
        else *it = x;
    }
    return sz(res);
}

// LIS Fast con Reconstrucción - O(N log N)
vi LISFastReconstruccion(const vi& arr) {
    int n = sz(arr);
    vi res, resIndex, padre(n, -1);

    rep(i, n) {
        int x = arr[i];
        auto it = lower_bound(all(res), x);
        int pos = it - res.begin();

        if (it == res.end()) {
            res.pb(x);
            resIndex.pb(i);
        } else {
            *it = x;
            resIndex[pos] = i;
        }

        if (pos > 0) padre[i] = resIndex[pos - 1];
    }

    vi lis;
    int idx = resIndex.back();
    while (idx != -1) {
        lis.pb(arr[idx]);
        idx = padre[idx];
    }
    reverse(all(lis));
    return lis;
}
```

Maximum Subarray

```
// Kadane's Algorithm - Maximum Subarray Sum O(N)
ll maximumSub(const vll& arr) {
    ll cur = 0, ans = -LINF;
    each(x, arr) {
        cur = max(x, cur + x);
        ckmax(ans, cur);
    }
    return ans;
}
```

Subset Sum Bitset

```
// Subset Sum con Bitset & Binary Splitting - O((S / 64) *
sum(log count))
// 1. Bitset acelera transiciones booleanas dividiendo entre
64.
// 2. Binary splitting agrupa elementos repetidos: si 'val'
aparece C veces,
//    se descompone en potencias de 2: 1*val, 2*val, 4*val,
..., R*val.

const int MAX_SUM = 500005;

// ─── 1. Bounded Subset Sum con Bitset Estandar
──────────────────────────────
bitset<MAX_SUM> subsetSum(const vi& arr, int target) {
    // 1. Contar frecuencias
    map<int, int> freq;
    each(x, arr) if (x <= target) freq[x]++;

    // 2. Descomposicion binaria de frecuencias
    vi items;
    for (auto& [val, count] : freq) {
        int c = min(count, target / val);
        for (int k = 1; k <= c; k *= 2) {
            items.pb(val * k);
            c -= k;
        }
        if (c > 0) items.pb(val * c);
    }
    // 3. DP con Bitset
    bitset<MAX_SUM> dp;
    dp[0] = 1;
    each(w, items) {
        dp |= (dp << w);
    }
    return dp;
}

// ─── 2. Bitset con Tamaño Dinamico (Template Recursivo)
──────────────────────
// Ejecuta con bitset<len> ajustado al target para no
desperdiciar operaciones
int reachable_max = 0;
template <int len = 1>
void dynamicSubsetSum(int target, const vi& items) {
    if (target >= len) {
        dynamicSubsetSum<min(len * 2, MAX_SUM)>(target,
items);
        return;
    }
    bitset<len> dp;
    dp[0] = 1;
    each(x, items) dp |= (dp << x);

    for (int i = target; i >= 0; --i) {
        if (dp[i]) {
            reachable_max = i;
            break;
        }
    }
}

// Ejemplo de uso: maxima suma posible <= target con binary
splitting
int getMaxSubsetSum(const vi& valores, int target) {
    map<int, int> freq;
    each(x, valores) if (x <= target) freq[x]++;
    vi items;
    for (auto& [val, count] : freq) {
        int c = min(count, target / val);
        for (int k = 1; k <= c; k *= 2) {
            items.pb(val * k);
            c -= k;
```

```
        }
        if (c > 0) items.pb(val * c);
    }
    reachable_max = 0;
    dynamicSubsetSum(target, items);
    return reachable_max;
}


// ─── 3. Reconstruccion de Elementos Tomados
──────────────────────────────
vi reconstructSubsetSum(const vi& arr, int target) {
    int n = sz(arr);
    // Para reconstruccion exacta se pueden guardar bitsets
por capas o usar backtrack
    vector<bitset<MAX_SUM>> dp(n + 1);
    dp[0][0] = 1;
    rep(i, n) {
        dp[i + 1] = dp[i] | (dp[i] << arr[i]);
    }
    if (!dp[n][target]) return {}; // Imposible

    vi taken;
    int cur = target;
    for (int i = n - 1; i >= 0; --i) {
        if (cur >= arr[i] && dp[i][cur - arr[i]]) {
            taken.pb(arr[i]);
            cur -= arr[i];
        }
    }
    return taken;
}
```

Data Structures

Binary Lifting Weighted

```
struct BinLiftW {
  int n, K;
  vector<vector<pair<int,ll>>> g;
  vector<vector<int>> up;
  vector<vector<ll>> mx; // mx[k][v] = max edge weight on the
2^k jump from v
  vector<int> dep;
  ll merge(ll a, ll b) { return max(a,b); } // your merge here
  const ll ID = LLONG_MIN; // identity of merge
  BinLiftW(int n) : n(n), K(__lg(n)+1), g(n), up(K,vector<int>
(n)), mx(K,vector<ll>(n,LLONG_MIN)), dep(n) {}
  void add_edge(int a, int b, ll w) { g[a].push_back({b,w});
g[b].push_back({a,w}); }
  void build(int root = 0) {
    vector<int> s = {root};
    up[0][root] = root;
    while (!s.empty()) {
      int u = s.back(); s.pop_back();
      for (auto [v,w] : g[u]) if (v != up[0][u]) { up[0][v] =
u; mx[0][v] = w; dep[v] = dep[u]+1; s.push_back(v); }
    }
    for (int k = 1; k < K; k++)
      for (int v = 0; v < n; v++) {
        int m = up[k-1][v];
        up[k][v] = up[k-1][m];
        mx[k][v] = merge(mx[k-1][v],mx[k-1][m]);
      }
  }
  ll query(int a, int b) { // aggregate over edges of the path
a-b
    ll res = ID;
    if (dep[a] < dep[b]) swap(a,b);
    for (int k = 0; k < K; k++)
      if ((dep[a]-dep[b])>>k & 1) { res = merge(res,mx[k][a]);
a = up[k][a]; }
    if (a == b) return res;
    for (int k = K-1; k >= 0; k--)
      if (up[k][a] != up[k][b]) {
        res = merge(res,merge(mx[k][a],mx[k][b]));
        a = up[k][a]; b = up[k][b];
      }
    return merge(res,merge(mx[0][a],mx[0][b]));
  }
};
```

Binary Lifting

```
struct BinLift {
  int n, K;
  vector<vector<int>> g, up;
```

```
  vector<int> dep;
  BinLift(int n) : n(n), K(__lg(n)+1), g(n), up(K,vector<int>
(n)), dep(n) {}
  void add_edge(int a, int b) { g[a].push_back(b);
g[b].push_back(a); }
  void build(int root = 0) {
    vector<int> s = {root};
    up[0][root] = root; // root is its own parent
    while (!s.empty()) { // iterative dfs
      int u = s.back(); s.pop_back();
      for (int v : g[u]) if (v != up[0][u]) { up[0][v] = u;
dep[v] = dep[u]+1; s.push_back(v); }
    }
    for (int k = 1; k < K; k++)
      for (int v = 0; v < n; v++) up[k][v] = up[k-1][up[k-1]
[v]];
  }
  int kth(int u, int k) { // k-th ancestor of u, root if k >
dep[u]
    for (int i = 0; i < K; i++) if (k>>i & 1) u = up[i][u];
    return u;
  }
  int lca(int a, int b) {
    if (dep[a] < dep[b]) swap(a,b);
    a = kth(a,dep[a]-dep[b]);
    if (a == b) return a;
    for (int k = K-1; k >= 0; k--)
      if (up[k][a] != up[k][b]) { a = up[k][a]; b = up[k][b];
}
    return up[0][a];
  }
  int dist(int a, int b) { return
dep[a]+dep[b]-2*dep[lca(a,b)]; }
};
```

DSU Rollback

```
struct DSURollback {
  int n, comps;
  vector<int> p, siz;
  vector<pair<int,int>> st; // history: (child root, parent
root), (-1,-1) if no merge
  DSURollback(int n) : n(n), comps(n), p(n), siz(n,1) {
iota(p.begin(),p.end(),0); }
  int find(int x) { while (p[x] != x) x = p[x]; return x; } //
no path compression
  bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) { st.push_back({-1,-1}); return false; }
    if (siz[a] < siz[b]) swap(a,b);
    p[b] = a; siz[a] += siz[b]; comps--;
    st.push_back({b,a});
    return true;
  }
  void rollback() { // undoes the last unite (even if it did
not merge)
    auto [b,a] = st.back(); st.pop_back();
    if (b < 0) return;
    p[b] = b; siz[a] -= siz[b]; comps++;
  }
  bool same(int a, int b) { return find(a) == find(b); }
  int size(int x) { return siz[find(x)]; }
};
```

DSU

```
struct DSU {
  int n;
  vector<int> p, siz;
  DSU(int n) : n(n), p(n), siz(n,1) {
iota(p.begin(),p.end(),0); }
  int find(int x) { return p[x] == x ? x : p[x] = find(p[x]);
}
  bool unite(int a, int b) { // false if already in the same
set
    a = find(a); b = find(b);
    if (a == b) return false;
    if (siz[a] < siz[b]) swap(a,b);
    p[b] = a; siz[a] += siz[b];
    return true;
  }
  bool same(int a, int b) { return find(a) == find(b); }
  int size(int x) { return siz[find(x)]; }
};
```

Fenwick Tree 2D Range

```
struct Fenwick2DRange {
  int n, m;
  vector<vector<ll>> b[4];
  Fenwick2DRange(int n, int m) : n(n), m(m) {
    for (auto& b : b) b.assign(n+1,vector<ll>(m+1));
  }
  void add(int x, int y, ll v) { // point update on the
difference array
    ll c[4] = {v, v*x, v*y, v*x*y};
    for (int i = x+1; i <= n; i += i&-i)
      for (int j = y+1; j <= m; j += j&-j)
        for (int k = 0; k < 4; k++) b[k][i][j] += c[k];
  }
  void update(int x1, int y1, int x2, int y2, ll v) { // [x1,
x2) x [y1, y2) += v
    add(x1,y1,v); add(x1,y2,-v); add(x2,y1,-v); add(x2,y2,v);
  }
  ll pre(int x, int y) { // sum of [0, x) x [0, y)
    ll s[4] = {};
    for (int i = x; i > 0; i -= i&-i)
      for (int j = y; j > 0; j -= j&-j)
        for (int k = 0; k < 4; k++) s[k] += b[k][i][j];
    return s[0]*x*y - s[1]*y - s[2]*x + s[3];
  }
  ll query(int x1, int y1, int x2, int y2) { // [x1, x2) x
[y1, y2)
    return pre(x2,y2) - pre(x1,y2) - pre(x2,y1) + pre(x1,y1);
  }
};
```

Fenwick Tree 2D

```
struct Fenwick2D {
  int n, m;
  vector<vector<ll>> t;
  Fenwick2D(int n, int m) : n(n), m(m), t(n+1,vector<ll>(m+1))
{}
  Fenwick2D(const vector<vector<ll>>& a) :
Fenwick2D(a.size(),a[0].size()) {
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++)
add(i,j,a[i][j]);
  }
  void add(int x, int y, ll v) {
    for (int i = x+1; i <= n; i += i&-i)
      for (int j = y+1; j <= m; j += j&-j) t[i][j] += v;
  }
  ll pre(int x, int y) { // sum of [0, x) x [0, y)
    ll s = 0;
    for (int i = x; i > 0; i -= i&-i)
      for (int j = y; j > 0; j -= j&-j) s += t[i][j];
    return s;
  }
  ll query(int x1, int y1, int x2, int y2) { // [x1, x2) x
[y1, y2)
    return pre(x2,y2) - pre(x1,y2) - pre(x2,y1) + pre(x1,y1);
  }
  void set(int x, int y, ll v) { add(x,y,v -
query(x,y,x+1,y+1)); }
};
```

Fenwick Tree Range

```
struct FenwickRange {
  int n;
  vector<ll> b1, b2;
  FenwickRange(int n) : n(n), b1(n+1), b2(n+1) {}
  void add(vector<ll>& b, int i, ll v) { for (i++; i <= n; i
+= i&-i) b[i] += v; }
  ll sum(vector<ll>& b, int i) { ll s = 0; for (; i > 0; i -=
i&-i) s += b[i]; return s; }
  void update(int l, int r, ll v) { // a[l..r-1] += v
    add(b1,l,v); add(b1,r,-v);
    add(b2,l,v*l); add(b2,r,-v*r);
  }
  ll pre(int i) { return sum(b1,i)*i - sum(b2,i); } // sum of
[0, i)
  ll query(int l, int r) { return pre(r) - pre(l); } // [l, r)
};
```

Fenwick Tree

```
struct Fenwick {
  int n;
  vector<ll> t;
  Fenwick(int n) : n(n), t(n+1) {}
  Fenwick(const vector<ll>& a) : n(a.size()), t(n+1) { // O(n)
```

```
    for (int i = 1; i <= n; i++) {
      t[i] += a[i-1];
      int j = i + (i&-i);
      if (j <= n) t[j] += t[i];
    }
  }
  void add(int i, ll v) { for (i++; i <= n; i += i&-i) t[i] +=
v; }
  ll pre(int i) { // sum of [0, i)
    ll s = 0;
    for (; i > 0; i -= i&-i) s += t[i];
    return s;
  }
  ll query(int l, int r) { return pre(r) - pre(l); } // [l, r)
  void set(int i, ll v) { add(i, v - query(i,i+1)); }
  int lower_bound(ll v) { // first i with pre(i+1) >= v,
(values >= 0), n if none
    int pos = 0;
    for (int k = 1<<__lg(n); k; k >>= 1)
      if (pos+k <= n && t[pos+k] < v) { pos += k; v -= t[pos];
}
    return pos;
  }
};
```

HLD

```
struct HLDNode { ll val = 0; };
struct HLDTree {
  int n;
  vector<HLDNode> t;
  vector<ll> lazy;
  HLDTree(int n) : n(n), t(4*n), lazy(4*n) {}
  HLDNode merge(const HLDNode& a, const HLDNode& b) { return
{a.val + b.val}; }
  void apply(int x, int l, int r, ll v) { t[x].val += v * (r-
l); lazy[x] += v; }
  void push(int x, int l, int r) {
    if (!lazy[x] || r-l == 1) return;
    int m=(l+r)/2; apply(2*x+1,l,m,lazy[x]);
apply(2*x+2,m,r,lazy[x]); lazy[x]=0;
  }
  void build(const vector<ll>& a, int x, int l, int r) {
    if (r-l==1) { t[x].val=a[l]; return; }
    int m=(l+r)/2; build(a,2*x+1,l,m); build(a,2*x+2,m,r);
t[x]=merge(t[2*x+1],t[2*x+2]);
  }
  void build(const vector<ll>& a) { build(a,0,0,n); }
  void update(int ql,int qr,ll v,int x,int l,int r) {
    if (qr<=l || r<=ql) return;
    if (ql<=l && r<=qr) { apply(x,l,r,v); return; }
    push(x,l,r); int m=(l+r)/2; update(ql,qr,v,2*x+1,l,m);
update(ql,qr,v,2*x+2,m,r); t[x]=merge(t[2*x+1],t[2*x+2]);
  }
  void update(int l,int r,ll v) { update(l,r,v,0,0,n); }
  HLDNode query(int ql,int qr,int x,int l,int r) {
    if (ql<=l && r<=qr) return t[x];
    push(x,l,r); int m=(l+r)/2;
    if (qr<=m) return query(ql,qr,2*x+1,l,m);
    if (ql>=m) return query(ql,qr,2*x+2,m,r);
    return
merge(query(ql,qr,2*x+1,l,m),query(ql,qr,2*x+2,m,r));
  }
  HLDNode query(int l,int r) { return query(l,r,0,0,n); }
};

struct HLD {
  int n;
  vector<vector<int>> g;
  vector<int> par, dep, siz, heavy, head, pos;
  HLDTree st;
  HLD(int n) : n(n), g(n), par(n,-1), dep(n), siz(n,1),
heavy(n,-1), head(n), pos(n), st(n) {}
  void add_edge(int a, int b) { g[a].push_back(b);
g[b].push_back(a); }
  void build(const vector<ll>& a, int root = 0) {
    vector<int> ord, s = {root};
    while (!s.empty()) { // preorder, sets parent and depth
      int u = s.back(); s.pop_back(); ord.push_back(u);
      for (int v : g[u]) if (v != par[u]) { par[v] = u; dep[v]
= dep[u]+1; s.push_back(v); }
    }
    for (int i = n-1; i > 0; i--) siz[par[ord[i]]] +=
siz[ord[i]];
    for (int i = n-1; i > 0; i--) {
      int u = ord[i], p = par[u];
      if (heavy[p] < 0 || siz[u] > siz[heavy[p]]) heavy[p] =
u;
```

```
    }
    int cur = 0; head[root] = root; s = {root};
    while (!s.empty()) { // heavy child is pushed last, so it
is visited right after u
      int u = s.back(); s.pop_back(); pos[u] = cur++;
      for (int v : g[u]) if (v != par[u] && v != heavy[u]) {
head[v] = v; s.push_back(v); }
      if (heavy[u] >= 0) { head[heavy[u]] = head[u];
s.push_back(heavy[u]); }
    }
    vector<ll> b(n);
    for (int i = 0; i < n; i++) b[pos[i]] = a[i];
    st.build(b);
  }
  void update_path(int a, int b, ll v) { // vertices on the
path a-b
    for (; head[a] != head[b]; a = par[head[a]]) {
      if (dep[head[a]] < dep[head[b]]) swap(a,b);
      st.update(pos[head[a]],pos[a]+1,v);
    }
    if (dep[a] > dep[b]) swap(a,b);
    st.update(pos[a],pos[b]+1,v);
  }
  HLDNode query_path(int a, int b) {
    HLDNode res; // identity of merge
    for (; head[a] != head[b]; a = par[head[a]]) {
      if (dep[head[a]] < dep[head[b]]) swap(a,b);
      res = st.merge(res,st.query(pos[head[a]],pos[a]+1));
    }
    if (dep[a] > dep[b]) swap(a,b);
    return st.merge(res,st.query(pos[a],pos[b]+1));
  }
  void update_subtree(int u, ll v) {
st.update(pos[u],pos[u]+siz[u],v); }
  HLDNode query_subtree(int u) { return
st.query(pos[u],pos[u]+siz[u]); }
  int lca(int a, int b) {
    for (; head[a] != head[b]; a = par[head[a]])
      if (dep[head[a]] < dep[head[b]]) swap(a,b);
    return dep[a] < dep[b] ? a : b;
  }
};

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   int n = 5;
//   HLD h(n);
//   h.add_edge(0,1); h.add_edge(0,2); h.add_edge(1,3);
h.add_edge(1,4);
//   h.build({1,2,3,4,5});
//   h.update_path(3,2,10);
//   cout << h.query_path(4,2).val << "\n";
//   cout << h.query_subtree(1).val << "\n";
// }
```

Ordered Set

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

template <class T>
using ordered_set = tree<
T,
  null_type,
  less<T>,                // use less_equal<T> if you want to
allow duplicates (see note below)
  rb_tree_tag,
  tree_order_statistics_node_update>;

// --- Example usage ---

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//
//   ordered_set<int> s;
//
//   s.insert(10);
//   s.insert(20);
//   s.insert(30);
//   s.insert(40);
//
```

```
//   // [1] Get element by index (0-based)
//   cout << "Element at index 2: " << *s.find_by_order(2) <<
"\n"; // -> 30
//
//   // [2] Get index of an element
//   cout << "Index of 30: " << s.order_of_key(30) << "\n"; //
-> 2
//
//   // [3] Lower bound behavior
//   cout << "Index where 25 would be inserted: " <<
s.order_of_key(25) << "\n"; // -> 2
//
//   // [4] Iterate through all elements
//   for (auto x : s) cout << x << " ";
//   cout << "\n";
//
//   // [5] Check if an element exists
//   if (s.find(20) != s.end())
//     cout << "20 exists in set\n";
// }
```

Persistent Segment Tree

```
struct Node {
    ll val = 0;
    int lc = 0, rc = 0; // children, 0 = null node
};
struct PerSegTree {
  int n;
  vector<Node> t;
  vector<int> root; // root[i] = root node of version i
  PerSegTree(int n, int cap = 0) : n(n), t(1), root(1,0) {
t.reserve(cap+1); }
  Node merge(const Node& a,const Node& b) {
    Node res;
    res.val = a.val + b.val; // your merge here
    return res;
  }
  int build(const vector<ll>& a, int l, int r) {
    int x = t.size(); t.emplace_back();
    if (r - l == 1) {t[x].val = a[l]; return x; }
    int m = (l+r)/2;
    int lc = build(a,l,m), rc = build(a,m,r);
    t[x] = merge(t[lc],t[rc]); t[x].lc = lc; t[x].rc = rc;
    return x;
  }
  int update(int p, int i, ll v, int l, int r) { // returns
the new node
    int x = t.size(); t.push_back(t[p]);
    if (r - l == 1) {t[x].val += v; return x; } // your apply
here (= v to assign)
    int m = (l+r)/2;
    if (i < m) {int c = update(t[p].lc,i,v,l,m); t[x].lc = c;
}
    else       {int c = update(t[p].rc,i,v,m,r); t[x].rc = c;
}
    int lc = t[x].lc, rc = t[x].rc;
    t[x] = merge(t[lc],t[rc]); t[x].lc = lc; t[x].rc = rc;
    return x;
  }
  Node query(int ql, int qr, int x, int l, int r) {
    if (ql <= l && r <= qr) return t[x];
    int m = (l + r) / 2;
    if (qr <= m) return query(ql,qr,t[x].lc,l,m);
    if (ql >= m) return query(ql,qr,t[x].rc,m,r);
    return merge(query(ql,qr,t[x].lc,l,m),
                 query(ql,qr,t[x].rc,m,r));
  }
  int kth(int a, int b, ll k) { // k-th smallest (0 idx) in
the count difference of roots b and a
    int l = 0, r = n;
    while (r - l > 1) {
      int m = (l+r)/2;
      ll c = t[t[b].lc].val - t[t[a].lc].val;
      if (k < c) {a = t[a].lc; b = t[b].lc; r = m; }
      else       {k -= c; a = t[a].rc; b = t[b].rc; l = m; }
    }
    return l;
  }
  void build(const vector<ll>& a){ root[0] = build(a,0,n); }
  int update(int ver, int i, ll v){
root.push_back(update(root[ver],i,v,0,n)); return
root.size()-1; } // new version id
  Node query(int ver, int l, int r){ return
query(l,r,root[ver],0,n); }
};

// Ejemplo de uso (main comentado para compilar con la
```

```
plantilla):
// int main() {
//   vector<ll> values = {2, 1, 3, 4};
//   PerSegTree pst(values.size());
//   pst.build(values);                  // version 0
//   int version1 = pst.update(0, 1, 5); // new version, index
1 += 5
//   cout << pst.query(0, 0, 4).val << "\n";
//   cout << pst.query(version1, 0, 4).val << "\n";
// }
```

Segment Tree Lazy

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node {
    ll val = 0;
};
struct SegTree {
  int n;
  vector<Node> t;
  vector<ll> lz;
  SegTree(int n) : n(n), t(4*n), lz(4*n) {}
  Node merge(const Node& a,const Node& b) {
    Node res;
    res.val = a.val + b.val;
    return res;
  }
  void apply(int x, int l, int r, ll v) {
    t[x].val += v*(r-l);
    lz[x] += v;
  }
  void push(int x, int l, int r) {
    if (!lz[x]) return;
    int m = (l+r)/2;
    apply(2*x+1,l,m,lz[x]);
    apply(2*x+2,m,r,lz[x]);
    lz[x] = 0;
  }
  void build(const vector<ll>& a, int x, int l, int r) {
    if (r - l == 1) {t[x].val = a[l]; return; }
    int m = (l+r)/2;
    build(a,2*x+1,l,m);
    build(a,2*x+2,m,r);
    t[x]=merge(t[2*x+1],t[2*x+2]);
  }
  void update(int ql, int qr, ll v, int x, int l, int r) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) {apply(x,l,r,v); return; }
    push(x,l,r);
    int m = (l+r)/2;
    update(ql,qr,v,2*x+1,l,m);
    update(ql,qr,v,2*x+2,m,r);
    t[x]=merge(t[2*x+1],t[2*x+2]);
  }
  Node query(int ql, int qr, int x, int l, int r) {
    if (ql <= l && r <= qr) return t[x];
    push(x,l,r);
    int m = (l + r) / 2;
    if (qr <= m) return query(ql,qr,2*x+1,l,m);
    if (ql >= m) return query(ql,qr,2*x+2,m,r);
    return merge(query(ql,qr,2*x+1,l,m),
                 query(ql,qr,2*x+2,m,r));
  }
  void build(const vector<ll>& a){ build(a,0,0,n); }
  void update(int l, int r, ll v){ update(l,r,v,0,0,n); }
  Node query(int l, int r){ return query(l,r,0,0,n); }
};
```

Segment Tree

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct Node {
    ll val = 0;
};
struct SegTree {
  int n;
  vector<Node> t;
  SegTree(int n) : n(n), t(4*n) {}
  Node merge(const Node& a,const Node& b) {
    Node res;
    return res;
  }
  void update(int i, ll v, int x, int l, int r) {
    if (r - l == 1) {t[x].val = v; return; }
```

```
    int m = (l+r)/2;
    if (i < m)update(i,v,2*x+1,l,m);
    else      update(i,v,2*x+2,m,r);
    t[x]=merge(t[2*x+1],t[2*x+2]);
  }
  Node query(int ql, int qr, int x, int l, int r) {
    if (ql <= l && r <= qr) return t[x];
    int m = (l + r) / 2;
    if (qr <= m) return query(ql,qr,2*x+1,l,m);
    if (ql >= m) return query(ql,qr,2*x+2,m,r);
    return merge(query(ql,qr,2*x+1,l,m),
                 query(ql,qr,2*x+2,m,r));
  }
  void update(int i, ll v){ update(i,v, 0,0,n); }
  Node query(int l, int r){ return query(l,r,0,0,n); }
};
```

Small To Large Map

```
#include <iostream>
#include <vector>
#include <map>

using namespace std;

const int MAXN = 200005;
vector<int> adj[MAXN];
int valores[MAXN];

// Estructura que guardará el estado (frecuencias, elementos
únicos, etc.) para cada subárbol.
// Puede ser map<int,int>, set<int>, o una estructura
personalizada.
map<int, int> subtree_map[MAXN];

void dfs(int u, int p = -1)
{
    int largest_child = -1;
    int max_size = -1;

    // PASO 1: Procesar hijos y encontrar el hijo "Pesado"
(Heavy Child)
    for (int v : adj[u])
    {
        if (v != p)
        {
            dfs(v, u);
            // Identificamos el hijo cuya estructura de datos
es la más grande
            if ((int)subtree_map[v].size() > max_size)
            {
                max_size = subtree_map[v].size();
                largest_child = v;
            }
        }
    }

    // PASO 2: Heredar el estado del hijo mayor en O(1)
    if (largest_child != -1)
    {
        // En tu código hacias: subtree[u] = subtree[largest]
        // std::swap hace exactamente lo mismo por debajo
(intercambia punteros)
        swap(subtree_map[u], subtree_map[largest_child]);
    }

    // PASO 3: Mezclar los hijos "Ligeros" (Light Children)
    for (int v : adj[u])
    {
        if (v != p && v != largest_child)
        {
            // Iteramos sobre el hijo pequeño y lo insertamos
en el grande (u)
            for (auto const &[val, count] : subtree_map[v])
            {
                // Aquí extraes o eliminas contribuciones
previas si el problema lo requiere

                // Añadimos el valor al map del nodo actual
                subtree_map[u][val] += count;

                // Aquí calculas las nuevas contribuciones
            }
            // Opcional: Limpiar el map del hijo para liberar
memoria
            subtree_map[v].clear();
        }
    }
```

```
    // PASO 4: Añadir el propio nodo actual (raíz de este
subárbol)
    int mi_valor = valores[u];
    subtree_map[u][mi_valor]++;

    // PASO 5: Guardar la respuesta para el nodo u
    // res[u] = ...
}
```

Small To Large Santi

```
vi colores, distinct;
vll suma;
vii pares;
vector<vector<int>> graph;
map<pair<int, int>, ll> res;
map<int, int> original;
vector<map<int, int> *> subtree;

void dfs(int i, int parent = -1)
{
    int largest = -1;
    vector<int> children;

    for (int node : graph[i])
    {
        if (node != parent)
        {
            dfs(node, i);
            children.push_back(node);
            if (largest == -1 || subtree[largest]->size() <
subtree[node]->size())
            {
                largest = node;
            }
        }
    }

    if (largest == -1)
    {
        subtree[i] = new map<int, int>();
        suma[i] = 0;
    }
    else
    {
        subtree[i] = subtree[largest];
        suma[i] = suma[largest];
    }

    for (int child : children)
    {
        if (child == largest)
            continue;

        for (const auto &[val, count] : *subtree[child])
        {
            int anterior = (*subtree[i])[val];
            ll quito = (ll)anterior * (ll)(original[val] -
anterior);
            suma[i] -= quito;

            (*subtree[i])[val] += count;
            int nuevo = (*subtree[i])[val];
            ll anado = (ll)nuevo * (ll)(original[val] -
nuevo);
            suma[i] += anado;
        }
    }

    int col = colores[i];
    int anterior = (*subtree[i])[col];
    ll quito = (ll)anterior * (ll)(original[col] - anterior);
    suma[i] -= quito;

    (*subtree[i])[col] += 1;
    int nuevo = (*subtree[i])[col];
    ll anado = (ll)nuevo * (ll)(original[col] - nuevo);
    suma[i] += anado;

    distinct[i] = subtree[i]->size();

    if (parent != -1)
    {
        res[{parent, i}] = suma[i];
        res[{i, parent}] = suma[i];
    }
}
```

```
void solve()
{
    int n;
    cin >> n;
    colores.resize(n);
    distinct.assign(n, 0);
    read(colores);
    original.clear();
    each(color, colores)
    {
        original[color]++;
    }
    suma.assign(n, 0);
    pares.resize(n - 1);
    graph.assign(n, vector<int>());
    subtree.assign(n, nullptr);
    res.clear();

    rep(i, n - 1)
    {
        int u, v;
        cin >> u >> v;
        u--;
        v--;
        pares[i] = {u, v};
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    dfs(0);
    for (int i = 0; i < n - 1; i++)
    {
        cout << res[pares[i]];
        if (i < n - 2)
            cout << " ";
    }
    cout << nl;
}

int main()
{
    ios int t = 1;
    while (t--)
    {
        solve();
    }
    return 0;
}
```

Small To Large Set

```
#include <iostream>
#include <vector>
#include <set>

using namespace std;

const int MAXN = 200005;
vector<int> adj[MAXN];
int color[MAXN];
int distinct_count[MAXN];

// Usamos arreglo de sets. Evitamos punteros para prevenir
Memory Leaks.
set<int> subtree_set[MAXN];

void dfs_small_to_large_set(int u, int p = -1)
{
    int largest_child = -1;
    int max_size = -1;

    // 1. Encontrar el hijo pesado
    for (int v : adj[u])
    {
        if (v != p)
        {
            dfs_small_to_large_set(v, u);
            if ((int)subtree_set[v].size() > max_size)
            {
                max_size = subtree_set[v].size();
                largest_child = v;
            }
        }
    }

    // 2. Heredar el set más grande en O(1)
    if (largest_child != -1)
```

```
    {
        swap(subtree_set[u], subtree_set[largest_child]);
    }

    // 3. Fusionar los hijos ligeros
    for (int v : adj[u])
    {
        if (v != p && v != largest_child)
        {
            // Inserción por rango: altamente optimizada en
C++
            subtree_set[u].insert(subtree_set[v].begin(),
subtree_set[v].end());

            // Liberar memoria del hijo procesado
            subtree_set[v].clear();
        }
    }

    // 4. Procesar el nodo actual
    subtree_set[u].insert(color[u]);

    // 5. Guardar la respuesta
    distinct_count[u] = subtree_set[u].size();
}
```

Sparse Table

```
struct SparseTable {
  int n;
  vector<vector<ll>> t;
  ll merge(ll a, ll b) {
    return min(a,b);
  }
  SparseTable(const vector<ll>& a) : n(a.size()) {
    int K = __lg(n) + 1;
    t.assign(K, vector<ll>(n));
    t[0] = a;
    for (int k=1; k<K; k++)
      for (int i = 0; i+(1<<k) <= n; i++)
        t[k][i] = merge(t[k-1][i], t[k-1][i+(1<<(k-1))]);
  }
  ll query(int l, int r) { // [l, r), ops (min, max, gcd, and,
or)
    int k = __lg(r-l);
    return merge(t[k][l], t[k][r-(1<<k)]);
  }
  ll query_log(int l, int r) { // [l, r), any associative op
    ll res = 1e18; // identity (0 for sum/xor)
    for (int k = __lg(r-l); k >= 0; k--)
      if (l + (1<<k) <= r) { res = merge(res, t[k][l]); l +=
1<<k; }
    return res;
  }
};
```

Two Pointers

```
// Two Pointers & Sliding Window Aggregation (SWAG)

// 1. Sliding Window Estandar - O(N)
// Ejemplo: Maxima longitud de subarray con condicion monotona
int slidingWindow(const vi& a, int k) {
    int n = sz(a), ans = 0, l = 0;
    // Estado de la ventana (ej. frecuencias, suma)
    // vi freq(MAX_VAL, 0); int distinct = 0;

    rep(r, n) {
        // add(a[r]);
        // while (!valid()) { remove(a[l++]); }
        ckmax(ans, r - l + 1);
    }
    return ans;
}

// 2. SWAG (Sliding Window Aggregation) - O(1) amortizado por
push/pop/query
// Permite operaciones asociativas NO reversibles (ej: min,
max, gcd, matrix mul)
template<class T>
struct SWAG {
    struct Node {
        T val, agg;
    };
    stack<Node> s1, s2;

    T op(T a, T b) { return min(a, b); } // Cambiar operacion
```

```
(gcd, max, +, etc.)

    bool empty() { return s1.empty() && s2.empty(); }
    int size() { return sz(s1) + sz(s2); }

    void push(T val) {
        T agg = s2.empty() ? val : op(s2.top().agg, val);
        s2.push({val, agg});
    }

    void pop() {
        if (s1.empty()) {
            while (!s2.empty()) {
                T val = s2.top().val;
                s2.pop();
                T agg = s1.empty() ? val : op(val,
s1.top().agg);
                s1.push({val, agg});
            }
        }
        s1.pop();
    }

    T query() {
        if (empty()) return 0; // valor neutro
        if (s1.empty()) return s2.top().agg;
        if (s2.empty()) return s1.top().agg;
        return op(s1.top().agg, s2.top().agg);
    }
};
```

Geometry

Convex Hull

```
#include <bits/stdc++.h>

using namespace std;

struct point {
  double x, y;
  point() { x = y = 0; }
  point(double _x, double _y) : x(_x), y(_y) {}
  bool operator == (point const& t) const {
    return x == t.x && y == t.y;
  }
};

int orientation(point a, point b, point c) {
  double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
  if (v < 0) return -1; // clockwise
  if (v > 0) return +1; // counter-clockwise
  return 0;
}

bool cw(point a, point b, point c, bool include_collinear) {
  int o = orientation(a, b, c);
  return o < 0 || (include_collinear && o == 0);
}
bool collinear(point a, point b, point c) { return
orientation(a, b, c) == 0; }

void convex_hull(vector<point>& a, bool include_collinear =
false) {
  point p0 = *min_element(a.begin(), a.end(), [](point a,
point b) {
      return make_pair(a.y, a.x) < make_pair(b.y, b.x);
      });
  sort(a.begin(), a.end(), [&p0](const point& a, const point&
b) {
      int o = orientation(p0, a, b);
      if (o == 0)
      return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
      < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
      return o < 0;
      });
  if (include_collinear) {
    int i = (int)a.size()-1;
    while (i >= 0 && collinear(p0, a[i], a.back())) i--;
    reverse(a.begin()+i+1, a.end());
  }

  vector<point> st;
  for (int i = 0; i < (int)a.size(); i++) {
    while (st.size() > 1 && !cw(st[st.size()-2], st.back(),
a[i], include_collinear))
      st.pop_back();
```

```
    st.push_back(a[i]);
  }

  if (include_collinear == false && st.size() == 2 && st[0] ==
st[1])
    st.pop_back();

  a = st;
}

double area(const vector<point>& fig) {
  double res = 0;
  for (unsigned i = 0; i < fig.size(); i++) {
    point p = i ? fig[i - 1] : fig.back();
    point q = fig[i];
    res += (p.x - q.x) * (p.y + q.y);
  }
  return fabs(res) / 2;
}


// example use case

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   double n,r;
//   cin >> n >> r;
//
//   vector<point> coords(n);
//   for(int i = 0; i < n; i++) {
//     cin >> coords[i].x >> coords[i].y;
//   }
//
//   auto dist = [](point &a, point&b) {
//     return sqrt(pow(a.x-b.x,2)+pow(a.y-b.y,2));
//   };
//
//   const double PI = 3.141516;
//   double cir = 2 * PI * r;
//   convex_hull(coords);
//
//   double per = 0;
//   coords.push_back(coords[0]);
//   for(int i = 1; i < coords.size(); i++) {
//     per +=  dist(coords[i-1],coords[i]);
//   }
//
//   int ans = round(per+cir);
//   cout << ans << endl;
// }
//
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(0);
//   int t = 1;
//   //cin >> t;
//   while(t--)
//     solve();
// }
```

Vector

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long

// Devuelve la forma canonica de un vector e inclinarlos a un
lado
// Ejemplo: (-12, 3) -> (4, -1)

pair<ll, ll> canon(pair<ll, ll> a)
{
    ll div = gcd(abs(a.first), abs(a.second));
    a.first /= div;
    a.second /= div;
    if (a.first < 0 || (a.first == 0 && a.second < 0))
    {
        a.first *= -1;
        a.second *= -1;
    }
    return a;
}

// Devuelve el vector perpendicular de un vector

pair<ll, ll> perp(pair<ll, ll> v)
```

```
{
    return canon({-v.second, v.first});
}
```

Graphs

2-SAT

```
#include <bits/stdc++.h>

using namespace std;

#define fst first
#define snd second
#define ll long long
#define endl '\n'

const int INF = 1 << 30;
const int MAXN = 1e5+5;

struct TwoSatSolver {
  int n_vars;
  int n_vertices;
  vector<vector<int>> adj, adj_t;
  vector<bool> used;
  vector<int> order, comp;
  vector<bool> assignment;

  TwoSatSolver(int _n_vars) : n_vars(_n_vars), n_vertices(2 *
n_vars), adj(n_vertices), adj_t(n_vertices), used(n_vertices),
order(), comp(n_vertices, -1), assignment(n_vars) {
    order.reserve(n_vertices);
  }
  void dfs1(int v) {
    used[v] = true;
    for (int u : adj[v]) {
      if (!used[u])
        dfs1(u);
    }
    order.push_back(v);
  }

  void dfs2(int v, int cl) {
    comp[v] = cl;
    for (int u : adj_t[v]) {
      if (comp[u] == -1)
        dfs2(u, cl);
    }
  }

  bool solve_2SAT() {
    order.clear();
    used.assign(n_vertices, false);
    for (int i = 0; i < n_vertices; ++i) {
      if (!used[i])
        dfs1(i);
    }

    comp.assign(n_vertices, -1);
    for (int i = 0, j = 0; i < n_vertices; ++i) {
      int v = order[n_vertices - i - 1];
      if (comp[v] == -1)
        dfs2(v, j++);
    }

    assignment.assign(n_vars, false);
    for (int i = 0; i < n_vertices; i += 2) {
      if (comp[i] == comp[i + 1])
        return false;
      assignment[i / 2] = comp[i] > comp[i + 1];
    }
    return true;
  }

  void add_disjunction(int a, bool na, int b, bool nb) {
    // na and nb signify whether a and b are to be negated
    a = 2 * a ^ na;
    b = 2 * b ^ nb;
    int neg_a = a ^ 1;
    int neg_b = b ^ 1;
    adj[neg_a].push_back(b);
    adj[neg_b].push_back(a);
    adj_t[b].push_back(neg_a);
    adj_t[a].push_back(neg_b);
  }

};
```

```
// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   int n,m;
//   cin >> n >> m;
//
//   TwoSatSolver solver(m);
//   for(int i = 0; i < n; i++) {
//     int topping1, topping2;
//     char op1, op2;
//     cin >> op1 >> topping1 >> op2 >> topping2;
//     bool no_good1 = op1 == '-';
//     bool no_good2 = op2 == '-';
//     topping1--,topping2--;
//     solver.add_disjunction(topping1,no_good1, topping2,
no_good2);
//   }
//
//   bool poss = solver.solve_2SAT();
//   if(!poss) {
//     cout << "IMPOSSIBLE" << endl;
//     return;
//   }
//
//   for(int i = 0; i < m; i++) {
//     if(solver.assignment[i]) {
//       cout << "+ ";
//     } else {
//       cout << "- ";
//     }
//   }
// }
//
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//   int t = 1;
//   //cin >> t;
//   while(t--)
//     solve();
// }
```

Bellman Ford

```
#include <bits/stdc++.h>

using namespace std;

#define fst first
#define snd second
#define ll long long

const ll INF = LONG_LONG_MAX;
const int MAXN = 1e5+5;

struct Edge {
  int u, v;
  ll weight;
};

int n,m;
vector<Edge> edges;

// Bellman-Ford algorithm to detect negative cycles and find
shortest paths
bool bellman(int src, vector<ll> &dist) {
  dist.assign(n + 1, INF);
  dist[src] = 0;
  for(int i = 2; i <= n; i++)
  {
    for (int j = 0; j < m; j++)
    {
      int u = edges[j].u;
      int v = edges[j].v;
      ll weight = edges[j].weight;
      if (dist[u]!=INF && dist[u] + weight < dist[v])
        dist[v] = dist[u] + weight;
    }
  }
  for (int i = 0; i < m; i++)
  {
    int u = edges[i].u;
    int v = edges[i].v;
    ll weight = edges[i].weight;
    // True if neg-cylce exists
    if (dist[u]!=INF && dist[u] + weight < dist[v])
```

```
      return true;
  }
  return false;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//   cin >> n >> m;
//   edges.resize(m);
//
//   for (int i = 0; i < m; i++) {
//     int a, b;
//     ll w;
//     cin >> a >> b >> w;
//     edges[i] = {a, b, w};
//   }
//
//   vector<ll> dist;
//
//   bool hasNegativeCycle = bellman(1, dist);
//
//   if (hasNegativeCycle) {
//     cout << "negative-weight cycle\n";
//   } else {
//     if (dist[n] == INF) {
//       cout << "no path from 1 to " << n << "\n";
//     } else {
//       cout << "shortest path from 1 to " << n << " is: " <<
dist[n] << "\n";
//     }
//   }
// }
```

Cd

```
#include "bits/stdc++.h"
using namespace std;

using ll = long long;
using ld = long double;
#define sz(v) ((int)((v).size()))

const int MAXN = 100005;
```

```
int n;
vector<int> adj[MAXN];
int sub[MAXN];
bool removed[MAXN];
int parCentroid[MAXN];
vector<vector<pair<int,int>>> elements(MAXN);

vector<vector<pair<int,int>>> sufix(MAXN);
#define linea() cerr << "---------------------------" << '\n'
int dfs_size(int u, int p) {
    sub[u] = 1;
    for (int v : adj[u]) {
        if (v == p || removed[v]) continue;
        sub[u] += dfs_size(v, u);
    }
    return sub[u];
}

int find_centroid(int u, int p, int total) {
    for (int v : adj[u]) {
        if (v == p || removed[v]) continue;
        if (sub[v] > total / 2)
            return find_centroid(v, u, total);
    }
    return u;
}
void bfs(int node) {
    map<int, int> dist;
    queue<int> q;
    q.push(node);
    elements[node].push_back({node, 0});
    while (!q.empty()) {
        auto l = q.front();
        q.pop();
        for (auto& x : adj[l]) {
            if (dist[x] == 0 && !removed[x] && x != node) {
                dist[x] = dist[l] + 1;
                q.push(x);
                elements[node].push_back({x, dist[x]});
            }
        }
    }
```

```
    sort(elements[node].begin(), elements[node].end());

    sufix[node].resize(sz(elements[node]) + 1);

    pair<int, int> otro {1e9, 1e9};
    for (int i = sz(elements[node]) - 1; i >= 0; i--) {
        if (otro.second > elements[node][i].second) {
            otro = elements[node][i];
        }
        else if (otro.second == elements[node][i].second) {
            if (otro.first > elements[node][i].first) otro =
elements[node][i];
        }
        sufix[node][i] = otro;
    }
}
void decompose(int u, int p = -1) {
    int total = dfs_size(u, -1);
    int c = find_centroid(u, -1, total);
    bfs(c);
    removed[c] = true;

    parCentroid[c] = (p == -1 ? c : p); // root centroid
points to itself

    // Example: hook custom logic here (before recursing)
    // e.g., preprocess distances from c to other nodes in its
component

    for (int v : adj[c]) {
        if (!removed[v])
            decompose(v, c);
    }
}
int bs(vector<pair<int, int>>& v, int target) {
    int low = 0, high = sz(v) - 1;
    int ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (v[mid].first > target) {
            ans = mid;
            high = mid - 1;
        }
        else low = mid + 1;
    }
    return ans;
}
int dist(vector<pair<int, int>>& v, int target) {
    int low = 0, high = sz(v) - 1;
    int ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (v[mid].first == target) {
            return mid;
        }
        else if (v[mid].first < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

// Ejemplo de uso anterior (comentado para compilar con la
plantilla):
// int32_t main() {
//     ios::sync_with_stdio(false);
//     cin.tie(nullptr);
//     cin >> n;
//     for (int i = 0; i < n - 1; i++) {
//         int u, v;
//         cin >> u >> v;
//         adj[u].push_back(v);
//         adj[v].push_back(u);
//     }
//
//     decompose(1);
//     for (int i = 1; i < n; i++) {
//         int aux = i;
//         int ori = aux;
//         pair<int, int> res = {1e9, 1e9};
//         bool ya = false;
//         do {
//             int it = bs(elements[aux], ori);
//             if (it == -1) {
//                 aux = parCentroid[aux];
//                 linea();
//                 continue;
//             }
//             pair<int, int> ans = sufix[aux][it];
```

```
//             int it2 = dist(elements[aux], ori);
//             ans.second += elements[aux][it2].second;
//             if (ans.second < res.second) {
//                 res = ans;
//             }
//             else if (ans.second == res.second) {
//                 if (ans.first < res.first) {
//                     res = ans;
//                 }
//             }
//             aux = parCentroid[aux];
//             linea();
//             if (ya) break;
//             if (parCentroid[aux] == aux) ya = true;
//         } while (true);
//         cout << res.first << ' ';
//     }
//     cout << n << '\n';
//
//     return 0;
// }
```

Centroid Decomposition

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
using ll = long long;

const int MAXN = 200005;

// ---------- GLOBALS ----------
int n;
vector<int> adj[MAXN];
int sub[MAXN];         // subtree sizes
bool removed[MAXN];    // centroid removed flag
int parCentroid[MAXN]; // parent in centroid tree

int dfs_size(int u, int p) {
    sub[u] = 1;
    for (int v : adj[u]) {
        if (v == p || removed[v]) continue;
        sub[u] += dfs_size(v, u);
    }
    return sub[u];
}

int find_centroid(int u, int p, int total) {
    for (int v : adj[u]) {
        if (v == p || removed[v]) continue;
        if (sub[v] > total / 2)
            return find_centroid(v, u, total);
    }
    return u;
}

void decompose(int u, int p = -1) {
    int total = dfs_size(u, -1);
    int c = find_centroid(u, -1, total);
    removed[c] = true;

    parCentroid[c] = (p == -1 ? c : p); // root centroid
points to itself

    // Example: hook custom logic here (before recursing)
    // e.g., preprocess distances from c to other nodes in its
component

    for (int v : adj[c]) {
        if (!removed[v])
            decompose(v, c);
    }
}

// ---------- MAIN ----------

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//     cin >> n;
//     for (int i = 0; i < n - 1; i++) {
//         int u, v;
//         cin >> u >> v;
//         adj[u].push_back(v);
//         adj[v].push_back(u);
//     }
//
```

```
//     decompose(0);
//     cout << "Centroid decomposition parent array:\n";
//     for (int i = 0; i < n; i++)
//         cout << i << " -> " << parCentroid[i] << endl;
//
//     return 0;
// }
```

Dijkstra

```
const ll INF = LLONG_MAX / 4;
struct Dijkstra {
  int n;
  vector<vector<pair<int,ll>>> g;
  vector<ll> d;
  vector<int> par;
  Dijkstra(int n) : n(n), g(n) {}
  void add_edge(int a, int b, ll w) { g[a].push_back({b,w}); }
// directed
  void add_undirected(int a, int b, ll w) { add_edge(a,b,w);
add_edge(b,a,w); }
  void run(const vector<int>& src) { // multi-source
    d.assign(n,INF); par.assign(n,-1);

priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<>>
pq;
    for (int s : src) { d[s] = 0; pq.push({0,s}); }
    while (!pq.empty()) {
      auto [du,u] = pq.top(); pq.pop();
      if (du > d[u]) continue; // stale entry
      for (auto [v,w] : g[u])
        if (d[u]+w < d[v]) { d[v] = d[u]+w; par[v] = u;
pq.push({d[v],v}); }
    }
  }
  void run(int s) { run(vector<int>{s}); }
  vector<int> path(int t) { // s -> t, empty if unreachable
    vector<int> p;
    if (d[t] >= INF) return p;
    for (; t >= 0; t = par[t]) p.push_back(t);
    reverse(p.begin(),p.end());
    return p;
  }
};
```

Floyd Warshall

```
#include <bits/stdc++.h>

using namespace std;

#define fst first
#define snd second
#define ll long long

const int INF = 1 << 30;
const int MAXN = 1e3+5;

int n;
ll d[MAXN][MAXN];

void floyd_warshall() {
  for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
      for (int j = 1; j <= n; j++)
        if (d[i][k] + d[k][j] < d[i][j])
          d[i][j] = d[i][k] + d[k][j];
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   int s, t;
//   cin >> n;
//   cin >> s >> t;
//   for(int i = 1; i <= n; i++) {
//     for(int j = 1; j <= n; j++) {
//       cin >> d[i][j];
//       if(d[i][j] == -1) d[i][j] = INF;
//     }
//   }
//
//   if(d[s][t] == INF)
//     cout << "-1\n";
//   else
//    cout << d[s][t] << '\n';
// }
//
// int main() {
```

```
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//   int t = 1;
//   //cin >> t;
//   while(t--)
//     solve();
// }
```

Ford Fulkerson

```
#include <bits/stdc++.h>

using namespace std;

#define ll long long
#define endl '\n'

const ll INF = 1e18;

struct Dinic {
  struct Edge {
    int to, rev;
    ll cap;
  };

  int n;
  vector<vector<Edge>> g;
  vector<int> level, it;

  Dinic(int n) : n(n), g(n), level(n), it(n) {}

  void add_edge(int u, int v, ll cap) {
    g[u].push_back({v, (int)g[v].size(), cap});
    g[v].push_back({u, (int)g[u].size() - 1, 0});
  }

  bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q;
    level[s] = 0; q.push(s);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (auto &e : g[u]) {
        if (e.cap > 0 && level[e.to] == -1) {
          level[e.to] = level[u] + 1;
          q.push(e.to);
        }
      }
    }
    return level[t] != -1;
  }

  ll dfs(int u, int t, ll f) {
    if (u == t) return f;
    for (int &i = it[u]; i < (int)g[u].size(); i++) {
      Edge &e = g[u][i];
      if (e.cap > 0 && level[e.to] == level[u] + 1) {
        ll ret = dfs(e.to, t, min(f, e.cap));
        if (ret > 0) {
          e.cap -= ret;
          g[e.to][e.rev].cap += ret;
          return ret;
        }
      }
    }
    return 0;
  }

  ll max_flow(int s, int t) {
    ll flow = 0, f;
    while (bfs(s, t)) {
      fill(it.begin(), it.end(), 0);
      while ((f = dfs(s, t, INF)) > 0)
        flow += f;
    }
    return flow;
  }

  // Min-cut edges
  void print_mincut_edges(int s) {
    vector<bool> vis(n, false);
    queue<int> q;
    q.push(s);
    vis[s] = true;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (auto &e : g[u]) {
        if (e.cap > 0 && !vis[e.to]) {
```

```
          vis[e.to] = true;
          q.push(e.to);
        }
      }
    }
    for (int u = 0; u < n; ++u) {
      if (vis[u]) {
        for (auto &e : g[u]) {
          if (!vis[e.to] && e.cap == 0) {
            cout << u + 1 << " " << e.to + 1 << endl;
          }
        }
      }
    }
  }
};

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//   int n,m;
//   cin >> n >> m;
//   Dinic g(n);
//   for (int i = 0; i < m; ++i) {
//     int u, v, w;
//     cin >> u >> v;
//     w = 1;
//     u--;v--;
//     g.add_edge(u, v, w);
//   }
//   cout << g.max_flow(0,n-1) << endl;
//   g.print_mincut_edges(0);
// }
```

Functional Graph

```
#include <bits/stdc++.h>

using namespace std;

#define endl '\n'

const int MAXN = 2e5+5;
const int LOG = 22;

int up[MAXN][LOG];
bool visited[MAXN];
bool on_stack[MAXN];
int next_node[MAXN];
int cycle[MAXN];
int cycle_length[MAXN];
int depth[MAXN];

int cycle_idx[MAXN];

void mark_cycle(int u, int id) {
  cycle[u] = id;
  depth[u] = 0;
  cycle_length[id]++;
  cycle_idx[u] = cycle_length[id];
  if(!cycle[next_node[u]])
    mark_cycle(next_node[u], id);
}

void dfs(int u) {
  visited[u] = 1;
  on_stack[u] = 1;

  int v = next_node[u];
  if(on_stack[v]) {
    mark_cycle(v, v);
  } else if(!visited[v]){
    dfs(v);
  }

  if(!cycle[u])
    depth[u] = depth[v] + 1;

  on_stack[u] = 0;
}

int jumpto(int u, int k) {
  int v = u;
  for(int i = 0; i < LOG; i++) {
    if((1 << i) & k) {
      v = up[v][i];
    }
  }
```

```
  return v;
}


int distcycle(int u, int v) {
  if(cycle_idx[u] < cycle_idx[v]) {
    return cycle_idx[v] - cycle_idx[u];
  }
  int turn = (cycle_length[cycle[u]] - cycle_idx[u] + 1) %
cycle_length[cycle[u]];
  return turn + cycle_idx[v] - 1;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   int n,q;
//   cin >> n >> q;
//
//   for(int i = 1; i <= n; i++) {
//     cin >> next_node[i];
//     up[i][0] = next_node[i];
//   }
//
//   for(int j = 1; j < LOG; j++) {
//     for(int i = 1; i <= n; i++) {
//       up[i][j] = up[up[i][j-1]][j-1];
//     }
//   }
//
//   for(int i = 1; i <= n; i++) {
//     if(!visited[i]) dfs(i);
//   }
//
//   while(q--) {
//     int u,v;
//     cin >> u >> v;
//
//     int cyu = cycle[u];
//     int cyv = cycle[v];
//
//     if(cyu == 0) {
//       cyu = cycle[jumpto(u,depth[u])];
//     }
//     if(cyv == 0) {
//       cyv = cycle[jumpto(v,depth[v])];
//     }
//
//     int a = u;
//     int b = v;
//
//     if(cyu != cyv) {
//       cout << -1 << endl;
//       continue;
//     }
//
//     if(cycle[u] == 0 && cycle[v] == 0) {
//       if(depth[u] < depth[v]) {
//         cout << -1 << endl;
//         continue;
//       }
//       int diff = depth[u] - depth[v];
//       if(jumpto(u,diff) != v) {
//         cout << -1 << endl;
//       } else {
//         cout << diff << endl;
//       }
//       continue;
//     }
//
//     if(cycle[u] && cycle[v]) {
//       cout << distcycle(u,v) << endl;
//       continue;
//     }
//
//     if(cycle[u] == 0) {
//       int steps = depth[u];
//       steps += distcycle(jumpto(u,depth[u]), v);
//       cout << steps << endl;
//       continue;
//     }
//
//     if(cycle[v] == 0) {
//       cout << -1 <<endl;
//       continue;
//     }
//   }
// }
```

```
//
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//   int t = 1;
//   //cin >> t;
//   while(t--)
//     solve();
// }
```

Hungarian

```
#include <bits/stdc++.h>

using namespace std;

#define fst first
#define snd second
#define ll long long
#define ms(a, b) memset(a, b, sizeof(a))

const int INF = 1 << 30;
const int MAXN = 25;
const int MAXM = 25;

// hungarian weighted matching algo
// finds the MAX cost of MAX matching, to find mincost, add
edges as negatives
// Nodes are indexed from 1 on both sides
template<typename T>
struct KuhnMunkras { // n for left, m for right
  int n, m, match[MAXM];
  T g[MAXN][MAXM], lx[MAXN], ly[MAXM], slack[MAXM];
  bool vx[MAXN], vy[MAXM];

  void init(int n_, int m_) {
    ms(g,0); n = n_, m = m_;
  }

  void add(int u, int v, T w) {
    g[u][v] = w;
  }

  bool find(int x) {
    vx[x] = true;
    for (int y = 1; y <= m; ++y) {
      if (!vy[y]) {
        T delta = lx[x] + ly[y] - g[x][y];
        if (delta==0) {
          vy[y] = true;
          if (match[y] == 0 || find(match[y])) {
            match[y] = x;
            return true;
          }
        } else slack[y] = min(slack[y], delta);
      }
    }
    return false;
  }

  T matching() { // maximum weight matching
    fill(lx + 1, lx + 1 + n, numeric_limits<T>::lowest());
    ms(ly,0);
    ms(match,0);
    for (int i = 1; i <= n; ++i) {
      for (int j = 1; j <= m; ++j) lx[i] = max(lx[i], g[i]
[j]);
    }
    for (int k = 1; k <= n; ++k) {
      fill(slack + 1, slack + 1 + m,
numeric_limits<T>::max());
      while (true) {
        ms(vx,0);
        ms(vy,0);
        if (find(k)) break;
        else {
          T delta = numeric_limits<T>::max();
          for (int i = 1; i <= m; ++i) {
            if (!vy[i]) delta = min(delta, slack[i]);
          }
          for (int i = 1; i <= n; ++i) {
            if (vx[i]) lx[i] -= delta;
          }
          for (int i = 1; i <= m; ++i) {
            if (vy[i]) ly[i] += delta;
            if (!vy[i]) slack[i] -= delta;
          }
        }
```

```
      }
    }
    T result = 0;
    for (int i = 1; i <= n; ++i) result += lx[i];
    for (int i = 1; i <= m; ++i) result += ly[i];
    return result;
  }
};

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   int n, m;
//   cin >> n >> m;
//
//   KuhnMunkras<double> km;
//   km.init(n, m);
//
//   for (int i = 1; i <= n; ++i) {
//     for (int j = 1; j <= m; ++j) {
//       double c;
//       cin >> c;
//       km.add(i,j, -c);
//     }
//   }
//
//   double result = km.matching();
//   double total_cost = -result;
//   cout << total_cost << '\n';
// }
//
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//   int t = 1;
//   //cin >> t;
//   while(t--)
//     solve();
// }
```

Kosaraju Scc

```
#include <bits/stdc++.h>

using namespace std;

#define fst first
#define snd second
#define ll long long
#define ms(a, b) memset(a, b, sizeof(a))

const int INF = 1 << 30;
const int MAXN = 25;
const int MAXM = 25;

vector<bool> visited; // keeps track of which vertices are
already visited

// runs depth first search starting at vertex v.
// each visited vertex is appended to the output vector when
dfs leaves it.
void dfs(int v, vector<vector<int>> const& adj, vector<int>
&output) {
  visited[v] = true;
  for (auto u : adj[v])
    if (!visited[u])
      dfs(u, adj, output);
  output.push_back(v);
}

// input: adj -- adjacency list of G
// output: components -- the strongly connected components in
G
// output: adj_cond -- adjacency list of G^SCC (by root
vertices)
void strongly_connected_components(vector<vector<int>> const&
adj,
    vector<vector<int>> &components,
    vector<vector<int>> &adj_cond) {
  int n = (int)adj.size();
  components.clear();
  adj_cond.clear();

  // 1) Order the vertices by finish time in a first DFS.
  vector<int> order;
  visited.assign(n, false);
  for (int i = 0; i < n; i++) {
    if (!visited[i]) {
```

```
      dfs(i, adj, order);
    }
  }

  // 2) Reverse all edges to get the transpose graph.
  vector<vector<int>> adj_rev(n);
  for (int v = 0; v < n; v++) {
    for (int u : adj[v]) {
      adj_rev[u].push_back(v);
    }
  }

  // 3) Do a second DFS in descending order of finish times
(from step 1).
  visited.assign(n, false);
  reverse(order.begin(), order.end());

  // this array will store the "root" (representative) of each
SCC
  vector<int> roots(n, -1);

  for (auto v : order) {
    if (!visited[v]) {
      vector<int> component;
      dfs(v, adj_rev, component);
      // store the newly found component
      components.push_back(component);
      // the root is the minimum vertex in that component
(arbitrary choice)
      int root = *min_element(begin(component),
end(component));
      for (auto u : component) {
        roots[u] = root;
      }
    }
  }

  // 4) Build the condensation graph (SCC DAG).
  adj_cond.assign(n, {});
  for (int v = 0; v < n; v++) {
    for (auto u : adj[v]) {
      if (roots[v] != roots[u]) {
        adj_cond[roots[v]].push_back(roots[u]);
      }
    }
  }
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//
//   int n = 5, m = 5;
//   vector<vector<int>> adj(n);
//
//   vector<pair<int,int>> edges = {
//     {0,1},
//     {1,2},
//     {2,0},
//     {1,3},
//     {3,4}
//   };
//
//   for (auto &e : edges) {
//     adj[e.first].push_back(e.second);
//   }
//
//   vector<vector<int>> components, cond_graph;
//   strongly_connected_components(adj, components,
cond_graph);
//
//   for (auto &comp : components) {
//     sort(comp.begin(), comp.end());
//   }
//
//   cout << "Number of SCCs: " << (int)components.size() <<
"\n";
//   for (int i = 0; i < (int)components.size(); i++) {
//     cout << "SCC " << i << ": ";
//     for (auto &v : components[i]) {
//       cout << v << " ";
//     }
//     cout << "\n";
//   }
//
//   cout << "Condensation graph adjacency list:\n";
//   // We'll print only non-empty adjacency to see edges
//   for (int i = 0; i < (int)cond_graph.size(); i++) {
```

```
//     if (!cond_graph[i].empty()) {
//       cout << "root " << i << ": ";
//       for (auto &v : cond_graph[i]) {
//         cout << v << " ";
//       }
//       cout << "\n";
//     }
//   }
// }
//
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//   int t = 1;
//   //cin >> t;
//   while(t--)
//     solve();
// }
```

Kruskal

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
#define endl '\n'

struct Edge {
  ll u, v, weight;
  bool operator<(Edge const& other) const {
    return weight < other.weight;
  }
};

struct DSU {
  vector<int> parent, size;

  DSU(int n) {
    parent.resize(n);
    size.assign(n, 1);
    iota(parent.begin(), parent.end(), 0);
  }

  int find(int v) {
    if (v == parent[v]) return v;
    return parent[v] = find(parent[v]);
  }

  bool unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (size[a] < size[b]) swap(a, b);
    parent[b] = a;
    size[a] += size[b];
    return true;
  }
};

ll kruskal(int n, vector<Edge>& edges, vector<Edge>& result) {
  sort(edges.begin(), edges.end());
  DSU dsu(n);
  ll cost = 0;

  for (auto& e : edges) {
    if (dsu.unite(e.u, e.v)) {
      cost += e.weight;
      result.push_back(e);
    }
  }
  return cost;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//   ios::sync_with_stdio(false);
//   cin.tie(nullptr);
//
//   int n, m;
//   cin >> n >> m;
//
//   vector<Edge> edges;
//   edges.reserve(m);
//
//   for (int i = 0; i < m; i++) {
//     ll u, v, w;
//     cin >> u >> v >> w;
```

```
//     u--, v--;
//     edges.push_back({u, v, w});
//   }
//
//   vector<Edge> result;
//   ll total_cost = kruskal(n, edges, result);
//
//   if(result.size() == n-1) {
//     cout << total_cost << endl;
//   } else {
//     cout << "IMPOSSIBLE" << endl;
//   }
//
//   // Optional: print MST edges
//   // for (auto &e : result)
//   //     cout << e.u + 1 << " " << e.v + 1 << " " <<
e.weight << endl;
// }
```

Topological Sort

```
#include <bits/stdc++.h>

using namespace std;

#define fst first
#define snd second
#define ll long long

const int INF = 1 << 30;
const int MAXN = 1e5+5;

vector<int> adj[MAXN];
vector<int> topo;

int n,m;

void dfs(int u, vector<int> &visited) {
  visited[u] = 1;
  for (int &v : adj[u]) {
    if (!visited[v])
      dfs(v, visited);
    else if(visited[v] == 1){
      cout << "IMPOSSIBLE\n";
      exit(0);
    }
  }
  visited[u] = 2;
  topo.push_back(u);
}

void topological_sort() {
  topo.clear();
  vector<int> visited(n+1,0);

  for (int i = 1; i <= n; i++)
    if (!visited[i])
      dfs(i, visited);

  reverse(topo.begin(), topo.end());
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   cin >> n >> m;
//   for(int i = 0; i <= n; i++) adj[i].clear();
//   while(m--) {
//     int a,b;
//     cin >> a >> b;
//     adj[a].push_back(b);
//   }
//   topological_sort();
//   for (int x : topo) cout << x << " ";
//   cout << "\n";
// }
//
// int main() {
//   int t = 1;
//   // cin >> t;
//   while(t--)
//     solve();
// }
```

Math

Binary Exponentiation

```
#include <bits/stdc++.h>

#define ll long long

ll binpow(ll a, ll b) {
    if (b == 0)
        return 1;
    ll res = binpow(a, b / 2);
    if (b % 2)
        return res * res * a;
    else
        return res * res;
}

ll binpowIt(ll a, ll b) {
    ll res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}

int power(ll x, ll y, ll M)
{
    if (y == 0)
        return 1;

    int p = power(x, y / 2, M) % M;
    p = (p * p) % M;

    return (y % 2 == 0) ? p : (x * p) % M;
}
// inverse modular power(A, M - 2, M)
```

Expected Value

```
// Expected Value & Probability (Esperanza Matematica)
// 1. Linealidad de la Esperanza: E[X + Y] = E[X] + E[Y]
(siempre valido, aun dependientes)
// 2. Variable indicadora: E[I_A] = P(A)
// 3. Ensayos geometricos (primer exito con prob p): E = 1/p
// 4. Coupon Collector (recolectar n cupones): E = n *
sum_{i=1}^n (1/i) = n * H_n

// ─── Sistema Lineal para Esperanza en Grafos de Estados /
Markov ──────────────
// E[u] = 1 + sum_{v} P(u -> v) * E[v]
// Se transforma en: E[u] - sum_{v} P(u -> v) * E[v] = 1 (con
E[target] = 0)
// Resuelto con Eliminacion Gaussiana O(N^3)

const ld EPS = 1e-11;

// Retorna 1 si tiene solucion unica, 0 si no o infinitas.
Solucion en ans.
int gauss(vector<vector<ld>> a, vector<ld>& ans) {
    int n = sz(a), m = sz(a[0]) - 1;
    vi where(m, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int sel = row;
        for (int i = row; i < n; ++i)
            if (fabsl(a[i][col]) > fabsl(a[sel][col])) sel =
i;
        if (fabsl(a[sel][col]) < EPS) continue;
        for (int i = col; i <= m; ++i) swap(a[sel][i], a[row]
[i]);
        where[col] = row;

        for (int i = 0; i < n; ++i) {
            if (i != row) {
                ld c = a[i][col] / a[row][col];
                for (int j = col; j <= m; ++j) a[i][j] -=
a[row][j] * c;
            }
        }
        ++row;
    }
    ans.assign(m, 0);
    rep(i, m) {
        if (where[i] != -1) ans[i] = a[where[i]][m] /
a[where[i]][i];
    }
    rep(i, n) {
```

```
        ld sum = 0;
        rep(j, m) sum += ans[j] * a[i][j];
        if (fabsl(sum - a[i][m]) > EPS) return 0;
    }
    return 1;
}

// Construccion de matriz para n estados (0..n-1), absorbiendo
en target
// adj[u] = lista de {v, prob}
vector<ld> expectedSteps(int n, int target, const
vector<vector<pair<int, ld>>>& adj) {
    vector<vector<ld>> A(n, vector<ld>(n + 1, 0));
    rep(u, n) {
        if (u == target) {
            A[u][u] = 1;
            A[u][n] = 0; // E[target] = 0
            continue;
        }
        A[u][u] = 1.0;
        A[u][n] = 1.0; // sumatoria de costo/paso base = 1
        for (auto& edge : adj[u]) {
            int v = edge.ff;
            ld p = edge.ss;
            A[u][v] -= p;
        }
    }
    vector<ld> E;
    gauss(A, E);
    return E;
}
```

FFT

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long
const double PI = acos(-1);

namespace FFT {
    static vector<int> rev;
    static vector<complex<double>> raices{{0,0},{1,0}};

    void fft(vector<complex<double>>& a, bool invertido) {
        int n = a.size();

        if (rev.size() != n) {
            int k = __builtin_ctz(n); // log2(n)
            rev.assign(n, 0);
            for (int i = 0; i < n; i++) {
                rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k -
1));
            }
        }

        for (int i = 0; i < n; i++) {
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        }

        if (raices.size() < n) {
            int k = __builtin_ctz(raices.size());
            raices.resize(n);
            while ((1 << k) < n) {
                double angulo = 2 * PI / (1 << (k + 1));
                for (int i = 1 << (k - 1); i < (1 << k); i++)
{
                    raices[2 * i] = raices[i];
                    double ang = angulo * (2 * i + 1 - (1 <<
k));
                    raices[2 * i + 1] = complex<double>
(cos(ang), sin(ang));
                }
                k++;
            }
        }

        for (int len = 1; len < n; len = len << 1) {
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    complex<double> u = a[i + j];
                    complex<double> v = a[i + j + len] *
raices[len + j];
                    a[i + j] = u + v;
                    a[i + j + len] = u - v;
                }
            }
        }
```

```
        if (invertido) {
            reverse(a.begin() + 1, a.end());
            for (auto &x : a) x /= n;
        }
    }
} // namespace FFT

vector<ll> multiplicacion_polinomio(const vector<ll>& A, const
vector<ll>& B) {
    int n1 = A.size(), n2 = B.size();
    if (n1 == 0 || n2 == 0) return vector<ll>();

    int n = 1;
    while (n < n1 + n2 - 1) n = n << 1;
    vector<complex<double>> fa(n), fb(n);
    for (int i = 0; i < n1; i++) fa[i] = complex<double>
(A[i],0);
    for (int i = 0; i < n2; i++) fb[i] = complex<double>
(B[i],0);

    FFT::fft(fa, false);
    FFT::fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] *= fb[i];
    FFT::fft(fa, true);

    vector<ll> res(n1 + n2 - 1);
    for (int i = 0; i < res.size(); i++) {
        res[i] = llround(fa[i].real()); // Redondeo
    }
    return res;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//     vector<ll> A = {1, 2, 3}; // 1 + 2x + 3x^2
//     vector<ll> B = {2, 0, 1}; // 2 + 0x + 1x^2
//     vector<ll> C = multiplicacion_polinomio(A, B);
//     return 0;
// }
```

Matrix Exponentiation

```
#include <bits/stdc++.h>

using namespace std;

#define ll long long
const ll MOD = 998244353;

struct Matrix
{
  int rows, cols;
  vector<vector<ll>> mat;

  // Matriz rectangular
  Matrix(int r, int c) : rows(r), cols(c), mat(r, vector<ll>
(c, 0)) {}

  // Matriz cuadrada
  Matrix(int n) : rows(n), cols(n), mat(n, vector<ll>(n, 0))
{}

  // Identity matrix (matriz cuadrada)
  static Matrix identity(int n)
  {
    Matrix I(n);
    for (int i = 0; i < n; i++)
      I.mat[i][i] = 1;
    return I;
  }

  // Matrix multiplication (permite matrices rectangulares)
  Matrix operator*(const Matrix &other) const
  {
    Matrix product(rows, other.cols);
    for (int i = 0; i < rows; ++i)
      for (int k = 0; k < cols; ++k)
      {
        ll val = mat[i][k] % MOD;
        for (int j = 0; j < other.cols; ++j)
        {
          product.mat[i][j] = (product.mat[i][j] + val *
other.mat[k][j]) % MOD;
        }
```

```
      }
    return product;
  }

  // Matrix exponentiation (solo matrices cuadradas rows ==
cols)
  Matrix pow(ll power) const
  {
    Matrix result = identity(rows);
    Matrix base = *this;

    while (power > 0)
    {
      if (power & 1)
        result = result * base;
      base = base * base;
      power >>= 1;
    }
    return result;
  }
};

ll fibonacci(int n)
{
  if (n == 0)
    return 0;
  Matrix base(2);
  base.mat = {{1, 1}, {1, 0}};
  base = base.pow(n - 1);
  return base.mat[0][0]; // F(n)
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//   for(int i = 0; i < 25; i++)
//     cout << fibonacci(i) << " ";
// }

// Para crear Funciones con Recurrencias de la forma: F(n) =
a1*F(n-1) + a2*F(n-2) + ... + ad*F(n-d)
// La matriz inicial es: A = {{F(1), F(2), ... , F(d)}}
/* La matriz clave es:
   B =
   {
   {0 , 0 , 0, ... , ad},
   {1 , 0 , 0, ... , ad-1},
   {0 , 1 , 0, ... , ad-2},
   {. , . , ., ... , a2},
   {0 , 0 , 0,  1  , a1}
   }

   Se realiza:
   B = B.pow(n-d);
   A = A * B;

   En A queda: A = {{F(n-(d-1)), F(n-(d-2)), ..., F(n)}}
   Entonces F(n) = A[0][d-1];
*/
```

NTT

```
#include <bits/stdc++.h>
using namespace std;

#define ll long long

namespace NTT {
    // MOD debe ser un primo de la forma c * 2^k + 1
    // 998244353 = 119 * 2^23 + 1. Raíz primitiva G = 3.
    const ll MOD = 998244353;
    const ll G = 3; // Raíz primitiva para MOD

    ll fast_pow(ll a, ll b) {
        ll res = 1;
        a %= MOD;
        while (b > 0) {
            if (b & 1) res = (res * a) % MOD;
            a = (a * a) % MOD;
            b >>= 1;
        }
        return res;
    }

    ll inv(ll a) {
        return fast_pow(a, MOD - 2);
    }
```

```
    static vector<int> rev;
    static vector<ll> w;

    void precompute(int n) {
        if (rev.size() == n) return;
        int k = __builtin_ctz(n);
        rev.assign(n, 0);
        for (int i = 0; i < n; i++) {
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (k -
1));
        }

        w.resize(n);
        w[0] = 1;
        ll g_n = fast_pow(G, (MOD - 1) / n);
        for(int i = 1; i < n; i++) {
            w[i] = (w[i - 1] * g_n) % MOD;
        }
    }

    void fft(vector<ll>& a, bool invertido) {
        int n = a.size();
        precompute(n);

        for (int i = 0; i < n; i++) {
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        }

        for (int len = 1; len < n; len <<= 1) {
            int step = n / (2 * len);
            for (int i = 0; i < n; i += 2 * len) {
                for (int j = 0; j < len; j++) {
                    ll u = a[i + j];
                    ll v = (a[i + j + len] * w[j * step]) %
MOD;

                    a[i + j] = (u + v) % MOD;
                    a[i + j + len] = (u - v + MOD) % MOD;
                }
            }
        }

        if (invertido) {
            reverse(a.begin() + 1, a.end());
            ll n_inv = inv(n);
            for (auto &x : a) {
                x = (x * n_inv) % MOD;
            }
        }
    }
} // namespace NTT

vector<ll> multiplicacion_polinomio_mod(const vector<ll>& A,
const vector<ll>& B) {
    int n1 = A.size(), n2 = B.size();
    if (n1 == 0 || n2 == 0) return vector<ll>();

    int n = 1;
    while (n < n1 + n2 - 1) n = n << 1;

    vector<ll> fa(n), fb(n);
    for (int i = 0; i < n1; i++) fa[i] = (A[i] % NTT::MOD +
NTT::MOD) % NTT::MOD;
    for (int i = 0; i < n2; i++) fb[i] = (B[i] % NTT::MOD +
NTT::MOD) % NTT::MOD;

    NTT::fft(fa, false);
    NTT::fft(fb, false);

    for (int i = 0; i < n; i++) {
        fa[i] = (fa[i] * fb[i]) % NTT::MOD;
    }

    NTT::fft(fa, true);

    fa.resize(n1 + n2 - 1);
    return fa;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//     vector<ll> A = {1, 2, 3}; // 1 + 2x + 3x^2
//     vector<ll> B = {2, 0, 1}; // 2 + 0x + 1x^2
//     vector<ll> C = multiplicacion_polinomio_mod(A, B);
//     return 0;
// }
```

Pollard Rho

```
#include <bits/stdc++.h>
using namespace std;

using u64 = uint64_t;
using u128 = __uint128_t;

// Multiplicacion modular para int128
u64 binpower(u64 base, u64 e, u64 mod)
{
    u64 result = 1;
    base %= mod;
    while (e)
    {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

// Test de primalidad de Miller Rabin
bool check(u64 n, u64 a, u64 d, int s)
{
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++)
    {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
}
// Retorna true si es primo
bool MillerRabin(u64 n)
{
    if (n < 2)
        return false;
    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0)
    {
        d >>= 1;
        r++;
    }
    // Bases suficientes para verificar primalidad hasta 2^64
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    {
        if (n == a)
            return true;
        if (check(n, a, d, r))
            return false;
    }
    return true;
}

// Algoritmo Pollard Rho, devuelve un factor primo de n
u64 pollard_rho(u64 n)
{
    if (n % 2 == 0)
        return 2;
    if (MillerRabin(n))
        return n;

    u64 x = 2, y = 2, d = 1, c = 1;

    // Función pseudoaleatoria f(x) = x^2 + c
    auto f = [&](u64 x, u64 n, u64 c)
    {
        return ((u128)x * x + c) % n;
    };

    while (d == 1)
    {
        x = f(x, n, c);          // Tortuga (1 paso)
        y = f(f(y, n, c), n, c); // Liebre (2 pasos)

        u64 diff = x > y ? x - y : y - x;
        d = std::gcd(diff, n);

        if (d == n)
        {
            x = rand() % (n - 2) + 2;
            y = x;
```

```
            c = rand() % (n - 1) + 1;
            d = 1;
        }
    }
    return d;
}

// Función recursiva para obtener todos los factores primos y
sus exponentes
void factorize(u64 n, map<u64, int> &factors)
{
    if (n == 1)
        return;
    if (MillerRabin(n))
    {
        factors[n]++;
        return;
    }
    u64 divisor = pollard_rho(n);
    factorize(divisor, factors);
    factorize(n / divisor, factors);
}

int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    u64 n;
    cin >> n;
    // factors, obtiene todos los factores y exponentes
    map<u64, int> factors;
    factorize(n, factors);
    return 0;
}
```

Sieve

```
// Sieve of Eratosthenes - O(N log log N)

// ─── 1. Criba Estandar (Arreglo Booleano)
──────────────────────────────────
const int MAXN = 1e6 + 5;
bool is_prime[MAXN];
vi primes;

void sieve() {
    fill(is_prime, is_prime + MAXN, true);
    is_prime[0] = is_prime[1] = false;

    // Tachar pares primero
    for (int i = 4; i < MAXN; i += 2) is_prime[i] = false;

    // Solo revisar impares
    for (int i = 3; (ll)i * i < MAXN; i += 2) {
        if (!is_prime[i]) continue;
        for (int j = i * i; j < MAXN; j += 2 * i) {
            is_prime[j] = false;
        }
    }

    primes.pb(2);
    for (int i = 3; i < MAXN; i += 2) {
        if (is_prime[i]) primes.pb(i);
    }
}

// ─── 2. Criba con Bitset (Ultra Rapida, 8x menos memoria, N
hasta 1e7) ──────
const int MAX_BITSET = 10000005;
bitset<MAX_BITSET> bs;
vi primes_bs;

void bitsetSieve() {
    bs.set(); // Pone todos los bits en 1
    bs[0] = bs[1] = 0;
    for (int i = 4; i < MAX_BITSET; i += 2) bs[i] = 0;

    for (int i = 3; (ll)i * i < MAX_BITSET; i += 2) {
        if (bs[i]) {
            for (int j = i * i; j < MAX_BITSET; j += 2 * i) {
                bs[j] = 0;
            }
        }
    }

    primes_bs.pb(2);
    for (int i = 3; i < MAX_BITSET; i += 2) {
```

```
        if (bs[i]) primes_bs.pb(i);
    }
}
```

Strings

Aho Corasick

```
#include <bits/stdc++.h>
using namespace std;

// ALPHA is a compile-time constant: one contiguous allocation
for all nodes.
// toIdx maps a character to [0, ALPHA) — swap freely for any
alphabet.
template<int ALPHA>
struct AhoCorasick {
    function<int(char)> toIdx;
    vector<array<int, ALPHA>> go;
    vector<int> fail, dict, output;
    // output[v] = pattern index ending at v (-1 if none)

    AhoCorasick(function<int(char)> toIdx) : toIdx(toIdx) {
        go.emplace_back(); go.back().fill(-1);
        fail.push_back(0);
        dict.push_back(-1);
        output.push_back(-1);
    }

    int insert(const string& s, int patId) {
        int cur = 0;
        for (char c : s) {
            int ch = toIdx(c);
            if (go[cur][ch] == -1) {
                go[cur][ch] = go.size();
                go.emplace_back(); go.back().fill(-1);
                fail.push_back(0);
                dict.push_back(-1);
                output.push_back(-1);
            }
            cur = go[cur][ch];
        }
        output[cur] = patId;
        return cur;
    }

    // Build failure and dictionary links (call after all
insertions)
    void build() {
        queue<int> q;
        for (int c = 0; c < ALPHA; c++) {
            if (go[0][c] == -1) {
                go[0][c] = 0; // missing char at root loops
back to root
            } else {
                fail[go[0][c]] = 0;
                q.push(go[0][c]);
            }
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            dict[u] = (output[fail[u]] != -1) ? fail[u] :
dict[fail[u]];
            for (int c = 0; c < ALPHA; c++) {
                if (go[u][c] == -1) {
                    go[u][c] = go[fail[u]][c]; // compressed
goto
                } else {
                    fail[go[u][c]] = go[fail[u]][c];
                    q.push(go[u][c]);
                }
            }
        }
    }

    void search(const string& text, const vector<string>&
pats,
                function<void(int,int)> cb) {
        int cur = 0;
        for (int i = 0; i < (int)text.size(); i++) {
            cur = go[cur][toIdx(text[i])];
            for (int v = cur; v > 0; v = dict[v])
                if (output[v] >= 0)
                    cb(i - (int)pats[output[v]].size() + 1,
output[v]);
        }
    }
```

```
};

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   vector<string> patterns = {"he", "she", "his", "hers"};
//   AhoCorasick<26> ac([](char c) { return c - 'a'; });
//   for (int i = 0; i < (int)patterns.size(); ++i)
ac.insert(patterns[i], i);
//   ac.build();
//   ac.search("ahishers", patterns, [&](int pos, int id) {
//     cout << patterns[id] << " at " << pos << "\n";
//   });
// }
```

Bit Trie

```
struct BitTrie {
  static const int B = 30; // bits (use 62 with ll values)
  struct Node {
    int nxt[2] = {-1, -1}, cnt = 0;
  };
  vector<Node> t;
  BitTrie() : t(1) {}
  void insert(int v, int d = 1) { // d = 1 insert, d = -1
erase (the value must exist)
    int x = 0;
    for (int b = B-1; b >= 0; b--) {
      int k = v>>b & 1;
      if (t[x].nxt[k] < 0) { t[x].nxt[k] = t.size();
t.emplace_back(); }
      x = t[x].nxt[k];
      t[x].cnt += d;
    }
  }
  int maxXor(int v) { // max of v ^ u over u in the set (set
must be non-empty)
    int x = 0, res = 0;
    for (int b = B-1; b >= 0; b--) {
      int k = v>>b & 1;
      int y = t[x].nxt[k^1];
      if (y >= 0 && t[y].cnt > 0) { res |= 1<<b; x = y; }
      else x = t[x].nxt[k];
    }
    return res;
  }
  int countLess(int v, int lim) { // number of u in the set
with (v ^ u) < lim
    int x = 0, res = 0;
    for (int b = B-1; b >= 0 && x >= 0; b--) {
      int k = v>>b & 1, l = lim>>b & 1;
      if (l) {
        int y = t[x].nxt[k];
        if (y >= 0) res += t[y].cnt; // xor bit 0 < 1, whole
subtree counts
        x = t[x].nxt[k^1];
      } else x = t[x].nxt[k];
    }
    return res;
  }
};
```

Double Hash

```
#include <bits/stdc++.h>
using namespace std;

// Double hashing: use two independent hash functions
// to make collision probability negligible (~1/P1 * 1/P2)

struct DoubleHash {
    static const long long MOD1 = 1e9 + 7;
    static const long long MOD2 = 1e9 + 9;
    static const long long B1   = 131;
    static const long long B2   = 137;

    vector<long long> h1, h2, pw1, pw2;
    int n;

    DoubleHash(const string& s) : n(s.size()),
        h1(s.size()+1,0), h2(s.size()+1,0),
        pw1(s.size()+1,1), pw2(s.size()+1,1)
    {
        for (int i = 0; i < n; i++) {
            h1[i+1] = (h1[i] * B1 + s[i]) % MOD1;
            h2[i+1] = (h2[i] * B2 + s[i]) % MOD2;
            pw1[i+1] = pw1[i] * B1 % MOD1;
```

```
            pw2[i+1] = pw2[i] * B2 % MOD2;
        }
    }

    // Returns a pair (hash1, hash2) for s[l..r]
    pair<long long, long long> query(int l, int r) const {
        long long q1 = (h1[r+1] - h1[l] * pw1[r-l+1] % MOD1 +
MOD1) % MOD1;
        long long q2 = (h2[r+1] - h2[l] * pw2[r-l+1] % MOD2 +
MOD2) % MOD2;
        return {q1, q2};
    }

    bool equal(int l1, int r1, int l2, int r2) const {
        return query(l1, r1) == query(l2, r2);
    }
};

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   string s = "abracadabra";
//   DoubleHash h(s);
//   cout << h.query(0, 2).first << " " << h.query(0,
2).second << "\n";
//   cout << boolalpha << h.equal(0, 2, 7, 9) << "\n";
// }
```

Hash

```
#include <bits/stdc++.h>
using namespace std;

// Single polynomial hash of a string
// hash(s) = s[0]*B^(n-1) + s[1]*B^(n-2) + ... + s[n-1]*B^0
(mod P)
// Equivalent iterative: hash = hash*B + s[i]

const long long MOD = 1e9 + 7;
const long long BASE = 131;

long long hash_str(const string& s) {
    long long h = 0;
    for (char c : s) {
        h = (h * BASE + c) % MOD;
    }
    return h;
}

// Precompute prefix hashes and powers for O(1) substring
queries
struct HashTable {
    vector<long long> h, pw;
    long long mod, base;

    HashTable(const string& s, long long B = BASE, long long M
= MOD)
        : h(s.size() + 1, 0), pw(s.size() + 1, 1), mod(M),
base(B)
    {
        for (int i = 0; i < (int)s.size(); i++) {
            h[i + 1] = (h[i] * base + s[i]) % mod;
            pw[i + 1] = pw[i] * base % mod;
        }
    }

    // Hash of s[l..r] (0-indexed, inclusive)
    long long query(int l, int r) const {
        return (h[r + 1] - h[l] * pw[r - l + 1] % mod + mod) %
mod;
    }
};

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   string s = "competitive";
//   HashTable h(s);
//   cout << hash_str(s) << "\n";
//   cout << h.query(0, 3) << "\n"; // hash of s[0..3]
// }
```

KMP Automaton

```
#include <bits/stdc++.h>

using namespace std;
```

```
#define ll long long
#define endl '\n'

vector<int> prefix_function(string s) {
  int n = (int)s.length();
  vector<int> pi(n);
  for (int i = 1; i < n; i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j])
      j = pi[j-1];
    if (s[i] == s[j])
      j++;
    pi[i] = j;
  }
  return pi;
}

void compute_automaton(string s, vector<vector<int>>& aut) {
  s += '#';
  int n = s.size();
  vector<int> pi = prefix_function(s);
  aut.assign(n, vector<int>(26));
  for (int i = 0; i < n; i++) {
    for (int c = 0; c < 26; c++) {
      if (i > 0 && 'a' + c != s[i])
        aut[i][c] = aut[pi[i-1]][c];
      else
        aut[i][c] = i + ('a' + c == s[i]);
    }
  }
}

const int MAXN = 1e4+5;
const int MAXK = 1e3+5;
const int MOD = 1e9+7;


int n,k;
string pattern;
vector<vector<int>> aut;

long long binpow(long long a, long long b)
{
    long long result = 1;
    while (b) {
        if (b & 1)
            result = (result * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return result;
}

ll dp[MAXN][MAXK];
ll fdp(int pos, int state) {
  bool found = state == pattern.size();
  if(pos == n) {
    return  found;
  }

  ll &val = dp[pos][state];
  if(val != -1) return val;

  val = 0;

  if(found) {
    ll l = n-pos;
    return val = binpow(k, l);
  }

  for(int i = 0; i < k; i++) {
    val = (val + fdp(pos+1,aut[state][i])) % MOD;
  }

  return val;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// void solve() {
//   memset(dp, -1, sizeof dp);
//   cin >> n >> k >> pattern;
//
//   compute_automaton(pattern, aut);
//
//   cout << fdp(0,0) << endl;
// }
//
```

```
// int main() {
//   solve();
// }
```

KMP

```
#include <bits/stdc++.h>
using namespace std;

vector<int> buildFail(const string& P) {
    int m = P.size();
    vector<int> fail(m, 0);
    for (int i = 1, j = 0; i < m; i++) {
        while (j > 0 && P[i] != P[j]) j = fail[j - 1];
        if (P[i] == P[j]) j++;
        fail[i] = j;
    }
    return fail;
}

// ── Application 1: Count non-overlapping occurrences ──────────────────────
int countNonOverlapping(const string& T, const string& P) {
    auto fail = buildFail(P);
    int count = 0, j = 0, m = P.size();
    for (char c : T) {
        while (j > 0 && c != P[j]) j = fail[j - 1];
        if (c == P[j]) j++;
        if (j == m) { count++; j = 0; } // reset instead of fail[m-1]
    }
    return count;
}

// ── Application 2: Shortest period of a string ────────────────────────────
// The period of S is the smallest p such that S[i] == S[i % p] for all i.
// Using KMP: period = m - fail[m-1]   (if m % (m-fail[m-1]) == 0, else period = m)
int period(const string& S) {
    auto fail = buildFail(S);
    int m = S.size();
    int p = m - fail[m - 1];
    return (m % p == 0) ? p : m;
}

// ── Application 3: Concatenation search (good for multiple patterns) ──────
// Search for P in T by building the "combined" string P + '#' + T,
// then checking where fail[i] == m in the combined string.
vector<int> kmpConcat(const string& T, const string& P) {
    string S = P + '#' + T;     // '#' is a separator not in the alphabet
    auto fail = buildFail(S);
    int m = P.size();
    vector<int> matches;
    for (int i = m + 1; i < (int)S.size(); i++)
        if (fail[i] == m)
            matches.push_back(i - 2 * m); // convert back to T-index
    return matches;
}

// Ejemplo de uso anterior (main comentado para compilar con la plantilla):
// int main() {
//   string text = "ababa", pattern = "aba";
//   cout << countNonOverlapping(text, pattern) << "\n";
//   cout << period(text) << "\n";
//   for (int pos : kmpConcat(text, pattern)) cout << pos << " ";
//   cout << "\n";
// }
```

Rolling Hash

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1e9 + 7;
const long long BASE = 131;

long long power(long long base, long long exp, long long mod)
{
    long long result = 1;
    base %= mod;
```

```
    while (exp > 0) {
        if (exp & 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}

// Compute hash of every substring of length k in O(n) total
vector<long long> rolling_hash(const string& s, int k) {
    int n = s.size();
    if (n < k) return {};

    vector<long long> result;
    long long h = 0;
    long long base_k = power(BASE, k - 1, MOD);  // B^(k-1)

    // Initial window [0, k-1]
    for (int i = 0; i < k; i++)
        h = (h * BASE + s[i]) % MOD;
    result.push_back(h);

    for (int i = 1; i + k - 1 < n; i++) {
        // Remove s[i-1], add s[i+k-1]
        h = (h - (long long)s[i - 1] * base_k % MOD + MOD) %
MOD;
        h = (h * BASE + s[i + k - 1]) % MOD;
        result.push_back(h);
    }

    return result;
}

// Find all occurrences of pattern p in text t using rolling
hash
vector<int> rabin_karp(const string& t, const string& p) {
    int n = t.size(), m = p.size();
    long long ph = 0;
    for (char c : p) ph = (ph * BASE + c) % MOD;

    vector<long long> th = rolling_hash(t, m);
    vector<int> matches;
    for (int i = 0; i < (int)th.size(); i++)
        if (th[i] == ph)
            if (t.substr(i, m) == p)  // confirm match (avoid
hash collision)
                matches.push_back(i);
    return matches;
}

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   string text = "abacaba", pattern = "aba";
//   for (int pos : rabin_karp(text, pattern)) cout << pos <<
" ";
//   cout << "\n";
// }
```

String Hashing

```
#include <bits/stdc++.h>

using namespace std;

#define ll long long

const int MAXN = 1e5+5;
const int BASE = 31;
const int M = 1e9 + 7;

int pot[MAXN];

void compute_powers(int n) {
    int p_pow = 1;
    for (int i = 0; i < n; i++) {
        pot[i] = p_pow;
        p_pow = (p_pow * BASE) % M;
    }
}

vector<ll> compute_hash(string const& s) {
    int n = s.size();
    vector<ll> hash_values(n + 5);
    hash_values[0] = 0;
    for (int i = 1; i <= n; i++) {
        hash_values[i] = (hash_values[i - 1] * BASE + s[i -
1]) % M;
```

```
    }
    return hash_values;
}

ll get_hash(vector<ll> &hash_values, int a, int b) {
    return (hash_values[b] - (hash_values[a - 1] * pot[b - a +
1]) % M + M) % M;
}

// Ejemplo de uso anterior (comentado para pegar el algoritmo
con la plantilla):
// int main() {
//     string word = "ALLEY";
//     compute_powers(word.size());
//     vector<ll> hash = compute_hash(word);
//
//     for (int i = 1; i <= word.size(); i++) {
//         cout << hash[i] << ' ';
//     }
//     cout << endl;
//     for (int i = 1; i <= word.size(); i++) {
//         cout << word[i - 1] << ": " << get_hash(hash, i, i)
<< endl;
//     }
//
//     return 0;
// }
```

Suffix Array

```
#include <bits/stdc++.h>
using namespace std;

vector<int> suffix_array(const string& s) {
    int n = s.size();
    vector<int> sa(n), rank_(n), tmp(n);
    iota(sa.begin(), sa.end(), 0);
    for (int i = 0; i < n; i++) rank_[i] = s[i];
    for (int k = 1; k < n; k <<= 1) {
        auto key = [&](int i) -> pair<int,int> {
            return {rank_[i], i + k < n ? rank_[i + k] : -1};
        };
        sort(sa.begin(), sa.end(), [&](int a, int b){ return
key(a) < key(b); });
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            tmp[sa[i]] = tmp[sa[i-1]] + (key(sa[i]) !=
key(sa[i-1]));
        rank_ = tmp;
        if (rank_[sa[n-1]] == n-1) break;
    }
    return sa;
}

vector<int> kasai_lcp(const string& s, const vector<int>& sa)
{
    int n = sa.size();
    vector<int> rank_(n), lcp(n, 0);
    for (int i = 0; i < n; i++) rank_[sa[i]] = i;
    int h = 0;
    for (int i = 0; i < n; i++) {
        if (rank_[i] > 0) {
            int j = sa[rank_[i] - 1];
            while (i + h < n && j + h < n && s[i+h] == s[j+h])
h++;
            lcp[rank_[i]] = h;
            if (h > 0) h--;
        }
    }
    return lcp;
}

// ── 1. Pattern search: O(M log N)
──────────────────────────────────────

// Binary search on SA: all matches are a contiguous range
[lo, hi).
pair<int,int> search_range(const string& s, const vector<int>&
sa, const string& P) {
    int n = sa.size(), m = P.size();
    int lo = (int)(lower_bound(sa.begin(), sa.end(), 0, [&]
(int a, int) {
        return s.compare(a, m, P) < 0;
    }) - sa.begin());
    int hi = (int)(upper_bound(sa.begin(), sa.end(), 0, [&]
(int, int a) {
        return s.compare(a, m, P) > 0;
    }) - sa.begin());
    return {lo, hi};
```

```
}

// ── 2. Count distinct substrings ──────────────────────────
// Total substrings = N*(N+1)/2. Subtract duplicates counted
by LCP array.
long long distinct_substrings(const string& s) {
    auto sa  = suffix_array(s);
    auto lcp = kasai_lcp(s, sa);
    int n = s.size();
    long long total = (long long)n * (n + 1) / 2;
    for (int v : lcp) total -= v;
    return total;
}

// ── 3. Longest repeated substring ─────────────────────────
// The maximum value in the LCP array.
string longest_repeated(const string& s) {
    auto sa  = suffix_array(s);
    auto lcp = kasai_lcp(s, sa);
    int best = *max_element(lcp.begin(), lcp.end());
    int idx  = max_element(lcp.begin(), lcp.end()) -
lcp.begin();
    return best > 0 ? s.substr(sa[idx], best) : "";
}

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   string s = "banana";
//   vector<int> sa = suffix_array(s);
//   for (int pos : sa) cout << pos << " ";
//   cout << "\ndistinct substrings = " <<
distinct_substrings(s) << "\n";
//   cout << "longest repeated = " << longest_repeated(s) <<
"\n";
// }
```

Suffix Tree

```
#include <bits/stdc++.h>
using namespace std;
struct SuffixTree {
    struct Node {
        int ch[26], link, start, end, cnt;
        // end = INF for leaves (extends with each new char)
        Node(int s, int e) : link(0), start(s), end(e), cnt(0)
{
            fill(ch, ch + 26, -1);
        }
        int len(int phase) { return min(end, phase + 1) -
start; }
    };

    string s;
    vector<Node> t;
    int last, n;
    int activeNode, activeEdge, activeLen, remaining;
    int need;

    SuffixTree() {}

    void setSuffixLink(int v) {
        if (need != -1) t[need].link = v;
        need = v;
    }

    bool walkDown(int v) {
        if (activeLen >= t[v].len(n - 1)) {
            activeEdge += t[v].len(n - 1);
            activeLen  -= t[v].len(n - 1);
            activeNode  = v;
            return true;
        }
        return false;
    }

    void extend(int pos) {
        t[0].end = pos + 1;  // implicit: all leaves extend
        remaining++;
        need = -1;

        while (remaining > 0) {
            if (activeLen == 0) activeEdge = pos;
            int ae = s[activeEdge] - 'a';

            if (t[activeNode].ch[ae] == -1) {
```

```
                // Rule 2: create new leaf
                t[activeNode].ch[ae] = t.size();
                t.emplace_back(pos, INT_MAX);
                setSuffixLink(activeNode);
            } else {
                int nxt = t[activeNode].ch[ae];
                if (walkDown(nxt)) continue;

                if (s[t[nxt].start + activeLen] == s[pos]) {
                    // Rule 3: already exists — just extend
active len
                    activeLen++;
                    setSuffixLink(activeNode);
                    break;
                }

                // Rule 2: split
                int split = t.size();
                t.emplace_back(t[nxt].start, t[nxt].start +
activeLen);
                t[activeNode].ch[ae] = split;

                t[split].ch[s[pos] - 'a'] = t.size();
                t.emplace_back(pos, INT_MAX);
                t[nxt].start += activeLen;
                t[split].ch[s[t[nxt].start] - 'a'] = nxt;
                setSuffixLink(split);
            }

            remaining--;
            if (activeNode == 0 && activeLen > 0) {
                activeLen--;
                activeEdge = pos - remaining + 1;
            } else {
                activeNode = t[activeNode].link ?
t[activeNode].link : 0;
            }
        }
        n = pos + 1;
    }

    void build(const string& str) {
        s = str + "$";
        n = 0;
        t.clear();
        t.emplace_back(0, 0);  // root
        last = activeNode = activeLen = activeEdge = remaining
= 0;
        need = -1;
        for (int i = 0; i < (int)s.size(); i++) extend(i);
    }

    // DFS to collect suffix indices at leaves
    void dfs(int v, int depth, vector<int>& res) {
        bool leaf = true;
        for (int c = 0; c < 26; c++) {
            if (t[v].ch[c] == -1) continue;
            leaf = false;
            dfs(t[v].ch[c], depth + t[t[v].ch[c]].len(n - 1),
res);
        }
        if (leaf) {
            // leaf: suffix index = n - 1 - depth  (n includes
'$')
            res.push_back(n - 1 - depth);
        }
    }

    // Find all occurrences of pattern P in O(|P| + occ)
    vector<int> search(const string& p) {
        int v = 0, i = 0;
        while (i < (int)p.size()) {
            int c = p[i] - 'a';
            if (t[v].ch[c] == -1) return {};
            int nxt = t[v].ch[c];
            for (int j = t[nxt].start; j < t[nxt].end && i <
(int)p.size(); j++, i++)
                if (s[j] != p[i]) return {};
            v = nxt;
        }
        vector<int> res;
        dfs(v, 0, res);
        sort(res.begin(), res.end());
        return res;
    }
};

// --- Longest Repeated Substring ---
```

```
// DFS: track string depth (total chars from root).
// Deepest internal node (>= 2 children) gives LRS.
string longestRepeated(SuffixTree& st) {
    string best = "";
    function<void(int, int)> dfs = [&](int v, int depth) {
        int childCount = 0;
        for (int c = 0; c < 26; c++) {
            if (st.t[v].ch[c] == -1) continue;
            childCount++;
            int nxt = st.t[v].ch[c];
            dfs(nxt, depth + st.t[nxt].len(st.n - 1));
        }
        // internal node with depth > best
        if (childCount > 0 && depth > (int)best.size()) {
            // walk back up to find the string — simplified
via start pointer
            int start = st.t[v].start - (depth -
st.t[v].len(st.n - 1));
            best = st.s.substr(max(0, start), depth);
        }
    };
    dfs(0, 0);
    return best;  // removes trailing '$' if present
}

// --- Count distinct substrings ---
// Each edge contributes (edgeLen) new distinct substrings.
// Total = sum of all edge lengths in the tree.
long long countDistinct(SuffixTree& st) {
    long long total = 0;
    // node 0 is root (no parent edge), skip
    for (int v = 1; v < (int)st.t.size(); v++)
        total += st.t[v].len(st.n - 1);
    return total - 1;  // subtract 1 for the empty string
}

// --- Longest Common Substring of two strings ---
// Build suffix tree of S1 + '#' + S2 + '$'
// Find deepest internal node whose subtree has leaves from
both S1 and S2.
string longestCommon(const string& s1, const string& s2) {
    SuffixTree st;
    string combined = s1 + "#" + s2 + "$";
    st.build(combined);
    int n1 = s1.size();
    string best = "";

    // DFS: returns {has_s1_leaf, has_s2_leaf}
    function<pair<bool,bool>(int, int)> dfs = [&](int v, int
dep) -> pair<bool,bool> {
        bool has1 = false, has2 = false;
        bool leaf = true;

        for (int c = 0; c < 26; c++) {
            if (st.t[v].ch[c] == -1) continue;
            leaf = false;
            int nxt = st.t[v].ch[c];
            auto [h1, h2] = dfs(nxt, dep + st.t[nxt].len(st.n
- 1));
            has1 |= h1;
            has2 |= h2;
        }

        if (leaf) {
            int sufIdx = st.n - 1 - dep;
            has1 = (sufIdx < (int)n1);
            has2 = (sufIdx > (int)n1);  // > n1 skips the '#'
        }

        // internal node with leaves from both strings
        if (!leaf && has1 && has2 && dep > (int)best.size()) {
            // recover substring: edge ends at t[v].end, go
back 'dep' chars
            int endPos = st.t[v].end;
            int startPos = endPos - dep;
            if (startPos >= 0 && endPos <= (int)st.s.size())
                best = st.s.substr(startPos, dep);
        }
        return {has1, has2};
    };

    dfs(0, 0);
    return best;
}

// --- Pattern matching demo ---

// Ejemplo de uso anterior (main comentado para compilar con
```

```
la plantilla):
// int main() {
//   SuffixTree st;
//   st.build("banana");
//   cout << longestRepeated(st) << "\n";
//   cout << countDistinct(st) << "\n";
//   cout << longestCommon("banana", "ananas") << "\n";
//   for (int pos : st.search("ana")) cout << pos << " ";
//   cout << "\n";
// }
```

Trie

```
struct Trie {

  static const int A = 26; // alphabet size
  struct Node {
    int nxt[A], cnt = 0, end = 0; // cnt: words passing here,
end: words ending here
    Node() { memset(nxt, -1, sizeof nxt); }
  };
  vector<Node> t;
  Trie() : t(1) {}
  void insert(const string& s) {
    int x = 0;
    for (char c : s) {
      int k = c - 'a';
      if (t[x].nxt[k] < 0) { t[x].nxt[k] = t.size();
t.emplace_back(); }
      x = t[x].nxt[k];
      t[x].cnt++;
    }
    t[x].end++;
  }
  int find(const string& s) { // node of s, or -1
    int x = 0;
    for (char c : s) {
      x = t[x].nxt[c - 'a'];
      if (x < 0) return -1;
    }
    return x;
  }
  int count(const string& s) { int x = find(s); return x < 0 ?
0 : t[x].end; }  // exact matches
  int prefix(const string& s) { int x = find(s); return x < 0
? 0 : t[x].cnt; } // words with prefix s
  bool erase(const string& s) { // removes one copy, false if
absent
    if (!count(s)) return false;
    int x = 0;
    for (char c : s) { x = t[x].nxt[c - 'a']; t[x].cnt--; }
    t[x].end--;
    return true;
  }
};
```

Z Function

```
#include <bits/stdc++.h>
using namespace std;

vector<int> z_function(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    z[0] = n;
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = min(z[i - l], r - i);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}

// ── 1. Pattern matching: find all occurrences of P in T
──────────────────
// Concatenate P + '#' + T. Any position i in the T-part where
z[i] == |P|
// corresponds to a match at T[i - |P| - 1].
vector<int> find_all(const string& P, const string& T) {
    string s = P + '#' + T;
    auto z = z_function(s);
    int m = P.size();
    vector<int> matches;
    for (int i = m + 1; i < (int)s.size(); i++)
        if (z[i] == m) matches.push_back(i - m - 1);
    return matches;
}
```

```
// ── 2. Shortest period of a string ──────────────────────────
// The shortest period p satisfies: s[0..n-p-1] == s[p..n-1],
// i.e. z[p] + p == n (the suffix at p matches the prefix and
reaches the end).
int shortest_period(const string& s) {
    int n = s.size();
    auto z = z_function(s);
    for (int p = 1; p < n; p++)
        if (z[p] + p == n) return p;
    return n; // no period shorter than n
}

// ── 3. Longest border (prefix = suffix, proper) ─────────────
// A border of length k exists iff z[n-k] == k.
int longest_border(const string& s) {
    int n = s.size();
    auto z = z_function(s);
    for (int k = n - 1; k >= 1; k--)
        if (z[n - k] == k) return k;
    return 0;
}

// Ejemplo de uso anterior (main comentado para compilar con
la plantilla):
// int main() {
//   string text = "abacaba", pattern = "aba";
//   for (int pos : find_all(pattern, text)) cout << pos << "
";
//   cout << "\nperiod = " << shortest_period(text) << "\n";
// }
```

Otros

Plantilla

```
#include <bits/stdc++.h>
using namespace std;

// ─── Fast IO ──────────────────────────────────────────────────
──
#define ios ios::sync_with_stdio(false); cin.tie(nullptr);

// ─── Debug (solo activo en LOCAL, se elimina en juicio) ──────
#ifdef LOCAL
  #define dbg(x)  cerr << "[" << #x << "] = " << (x) << "\n"
  #define dbgv(v) cerr << "[" << #v << "] = "; for(auto _x:
(v)) cerr<<_x<<" "; cerr<<"\n"
  #define dbgm(m) for(auto& _r:(m)){for(auto _x:_r)cerr<<_x<<"
";cerr<<"\n";}
#else
  #define dbg(x)
  #define dbgv(v)
  #define dbgm(m)
#endif

// ─── Macros de iteración ──────────────────────────────────────
#define rep(i,n)    for(int i=0;i<(int)(n);i++)
#define rep1(i,n)   for(int i=1;i<=(int)(n);i++)
#define repr(i,n)   for(int i=(int)(n)-1;i>=0;i--)
#define each(x,a)   for(auto& x:(a))

// ─── Macros de contenedor ─────────────────────────────────────
#define all(s)   (s).begin(),(s).end()
#define rall(s)  (s).rbegin(),(s).rend()
#define pb       push_back
#define eb       emplace_back
#define sz(v)    ((int)((v).size()))

// ─── Shortcuts ────────────────────────────────────────────────
#define nl       '\n'
#define ff       first
#define ss       second
#define yesi     cout<<"Yes\n"
#define nosi     cout<<"No\n"

// ─── Typedefs ─────────────────────────────────────────────────
──
typedef long long      ll;
```

```
typedef long double    ld;
typedef pair<int,int>  ii;
typedef pair<ll,ll>    pll;
typedef vector<int>    vi;
typedef vector<ll>     vll;
typedef vector<ld>     vld;
typedef vector<ii>     vii;
typedef vector<pll>    vpll;
typedef vector<vi>     vvi;
typedef vector<vll>    vvll;
typedef vector<string> vs;
typedef vector<char>   vc;

// ─── Constantes
──────────────────────────────────────────────────────────
const ll LINF = (1LL << 62);

// ─── ckmin / ckmax
─────────────────────────────────────────────────────
template<class T> bool ckmin(T& a, const T& b){ return b<a ?
a=b,1:0; }
template<class T> bool ckmax(T& a, const T& b){ return b>a ?
a=b,1:0; }

// ─── Bit helper
──────────────────────────────────────────────────────────
int bit(int mask, int b){ return (mask >> b) & 1; }

// ─── RNG
──────────────────────────────────────────────────────────────
──────
mt19937
rng(chrono::steady_clock::now().time_since_epoch().count());
#define randint(a,b) uniform_int_distribution<int>(a,b)(rng)

// ─── Lectura rápida genérica
──────────────────────────────────────────
template<class T>           void read(T& x)           { cin
>> x; }
template<class F, class S>  void read(pair<F,S>& p)   { cin
>> p.first >> p.second; }
template<class T>           void read(vector<T>& v)   {
each(x,v) read(x); }
```

```
//
──────────────────────────────────────────────────────────────
──────────────

void solve() {

}

int main(){
    ios
    int t = 1;
    // cin >> t;
    while(t--){
      solve();
    }
    return 0;
}
```